

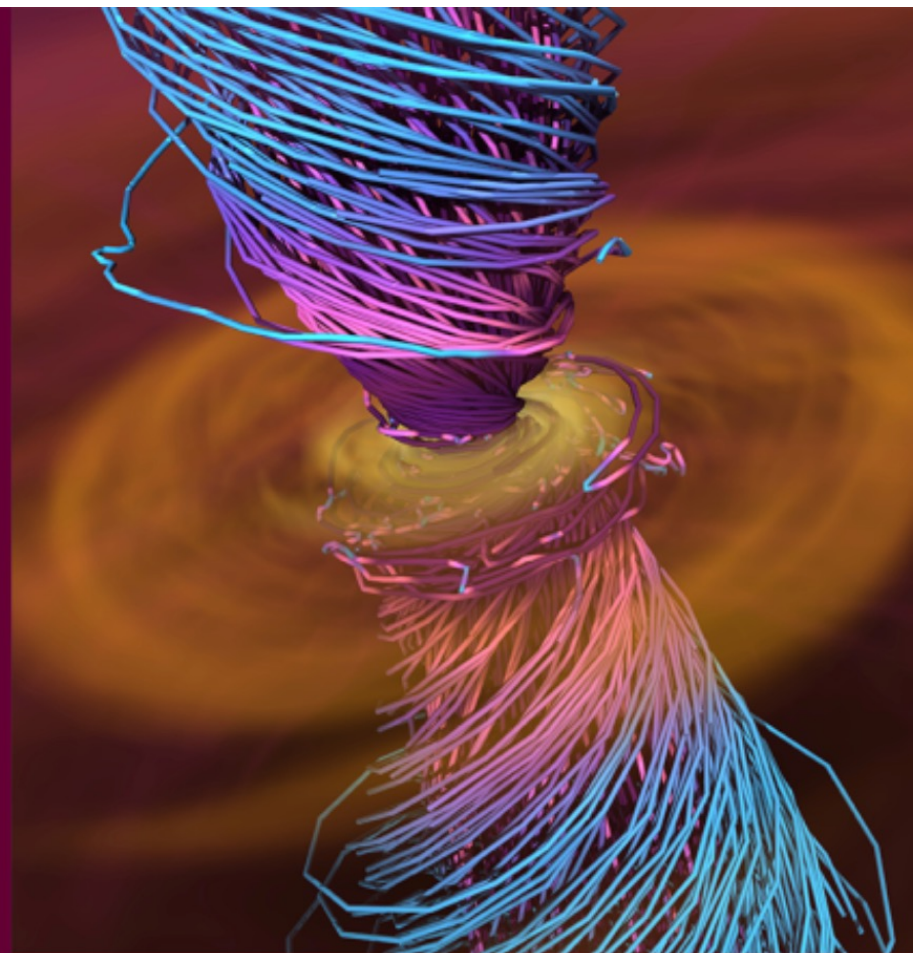
DEVELOPER SESSION

ALCF

Visualization at ALCF: Beyond the Basics

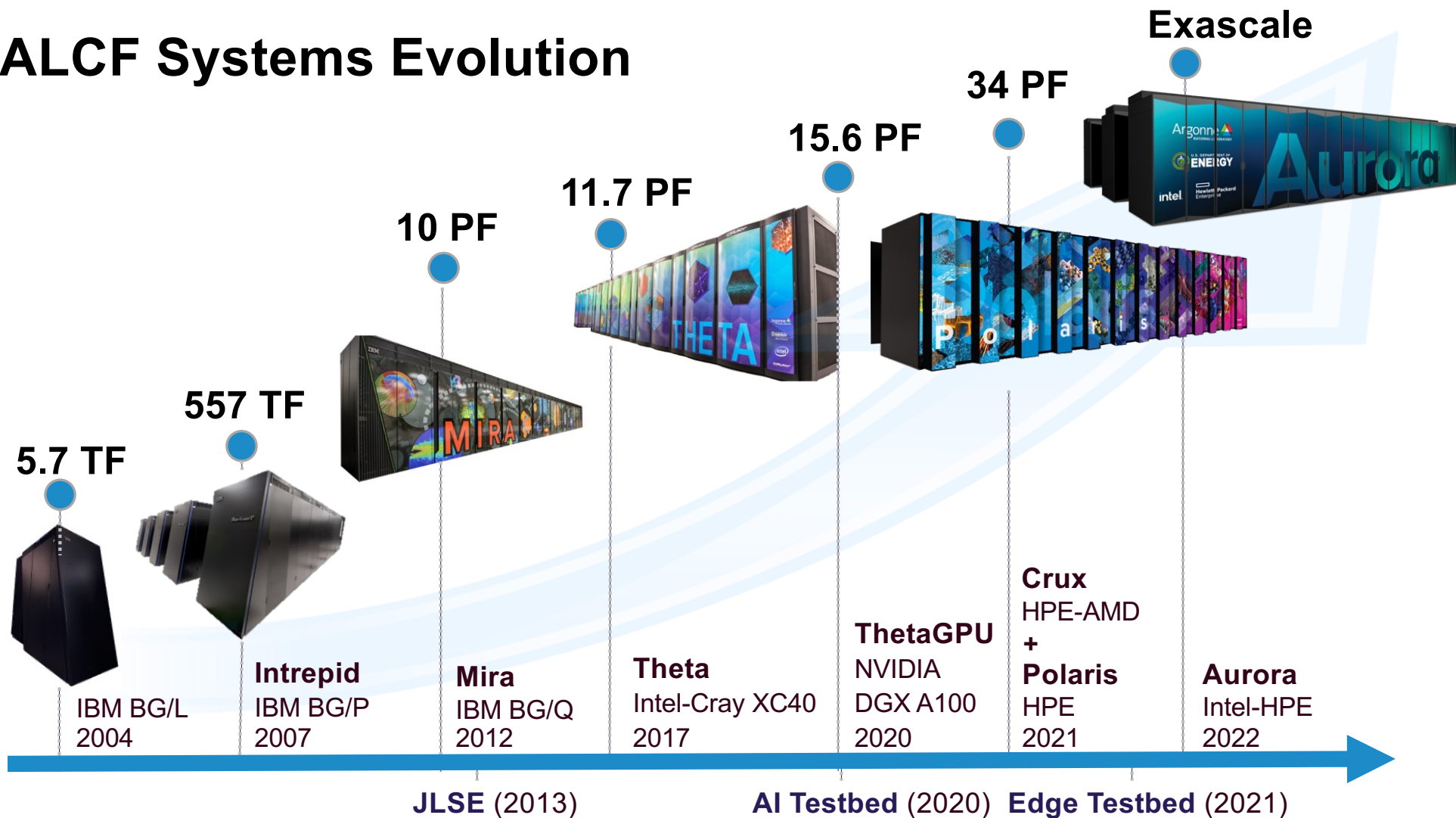
Virtual

April 22, 2026, 11:00am - 12:00pm CT

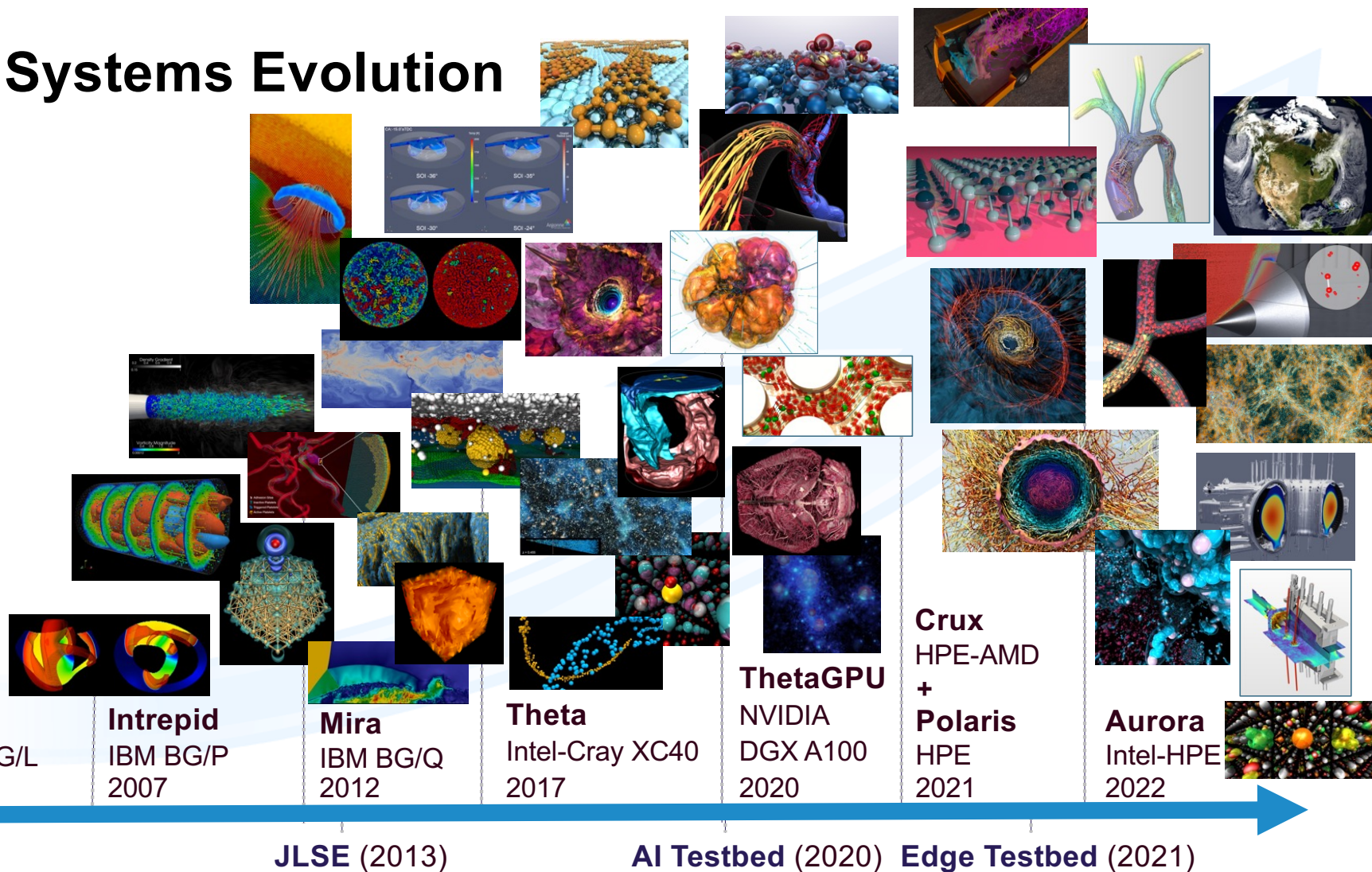


Joseph Insley, Victor Mateevitsi, Silvio Rizzi,
Janet Knowles, Mengjiao Han
ALCF Visualization Team

ALCF Systems Evolution



ALCF Systems Evolution

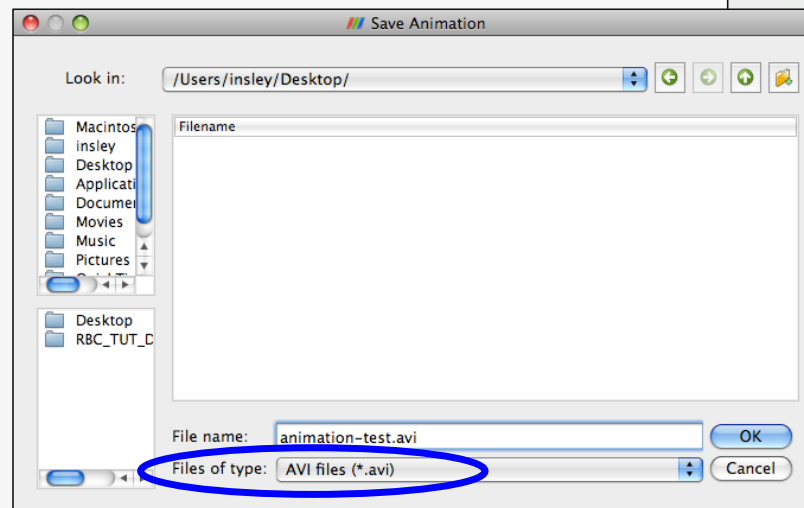
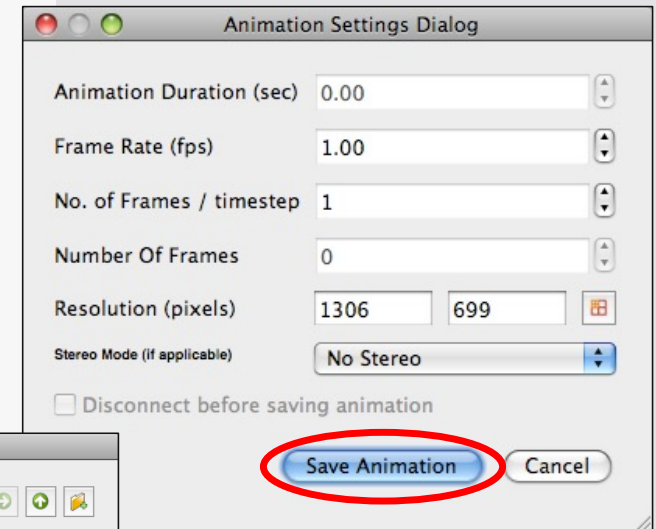


Save Screenshot

1. Choose File →  Save Screenshot...
2. Complete the following dialogs.

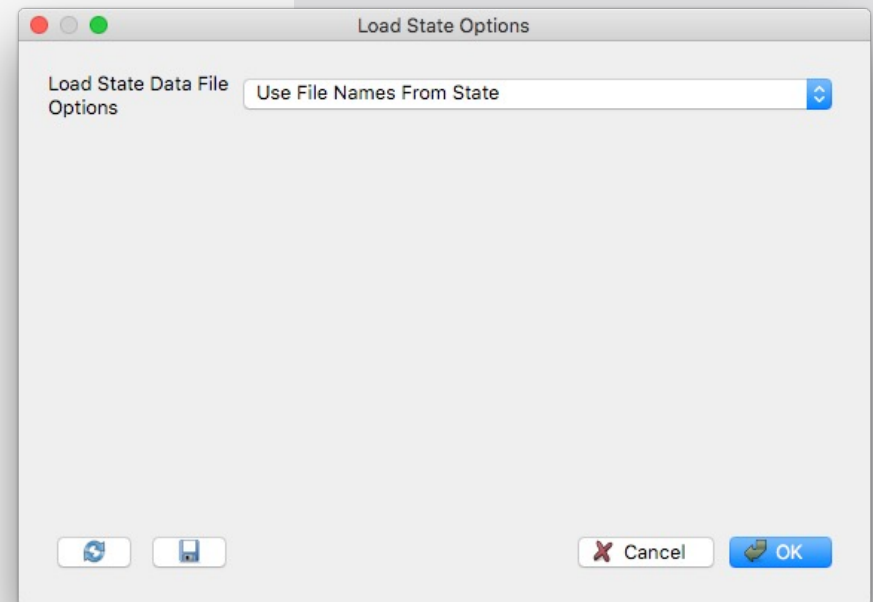
Save Animation

1. File-> Save Animation
2. Animation Settings Dialog: **Save Animation**
3. **Files of type:**
 - AVI
 - PNG



Saving ParaView State

1. Choose File → Save State...
 - .pvsm (for restoring state in interactive mode)
 - saved on the client side
2. Edit → Reset Session 
3. Choose File → Load State...
4. Load State Options
 1. Use File Names From State
 2. Search files under specified directory
 3. Choose File Names



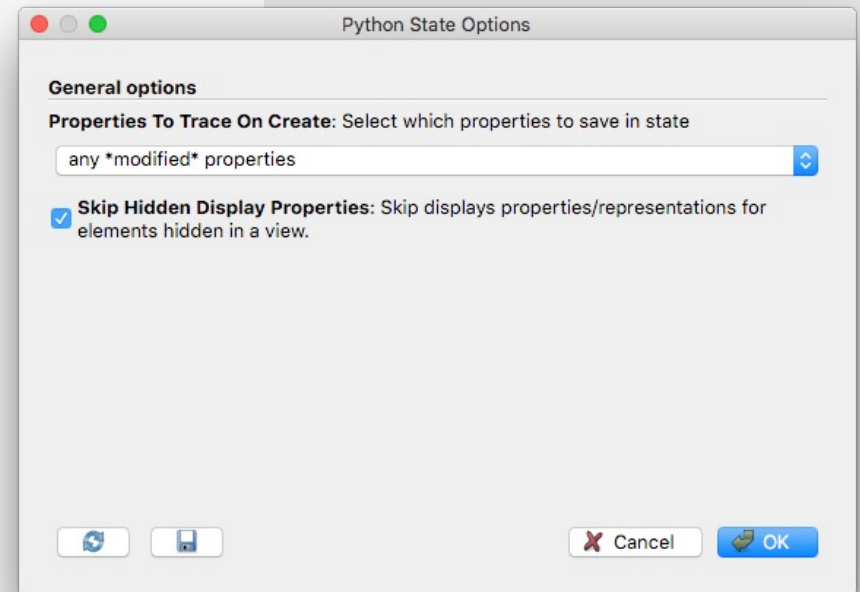
Saving ParaView State and Scripting

1. Choose File → Save State...

- .py (for use with pvbatch)
 - any *modified* properties
 - all properties
- saved on the client side

2. Edit .py script

- short example, loop over time steps, saving images in batch mode



Saving ParaView State and Scripting

At the top of the script:

```
import sys
start_frame=int(sys.argv[1])
num_frames=int(sys.argv[2])
IMAGE_DIR = "/path/to/where/we/should/save/images/
```

At the bottom of the script:

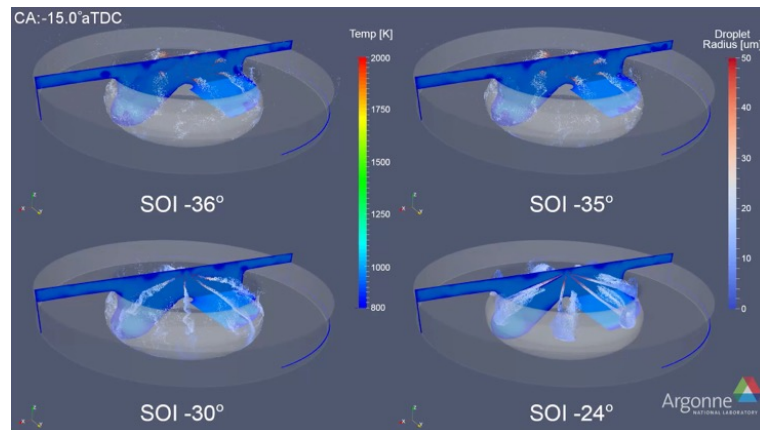
```
RenderView1.ViewSize = [1920, 1080]           # set resolution appropriately
time_vals = artery_step_0000vtu.TimestepValues # find data reader object
for i in range(start_frame, start_frame+num_frames):
    RenderView1.ViewTime = time_vals[i]
    RenderView1.StillRender()
    IMAGE_FILE="%s/frame_%04d.png" % (IMAGE_DIR, i)
    print ("saving: " + IMAGE_FILE)
    WriteImage(IMAGE_FILE)
```


Saving ParaView State and Scripting

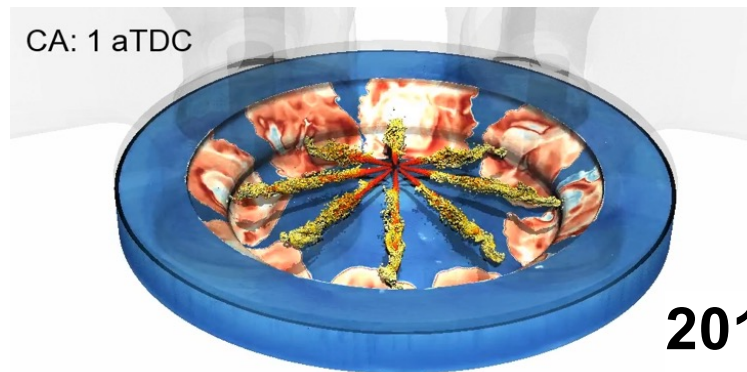
- Copy python script to Cooley
- Submit batch job to render animation
 - `qsub -n 1 -t 60 -A Comp_Perf_Workshop -q training2 /soft/visualization/paraview/v5.8.0/bin/pvbatch python_script.py 0 50`
 - `qsub -n 1 -t 60 -A Comp_Perf_Workshop -q training2 /soft/visualization/paraview/v5.8.0/bin/pvbatch python_script.py 50 50`
- Encode frames using ffmpeg
 - `soft add +ffmpeg`
 - `ffmpeg -r 25 -i /path/frame_%04d.png -r 25 -pix_fmt yuv420p movie_01.mp4`

Engineering / Combustion / Biofuels

PI: Sibendu Som, Argonne National Laboratory



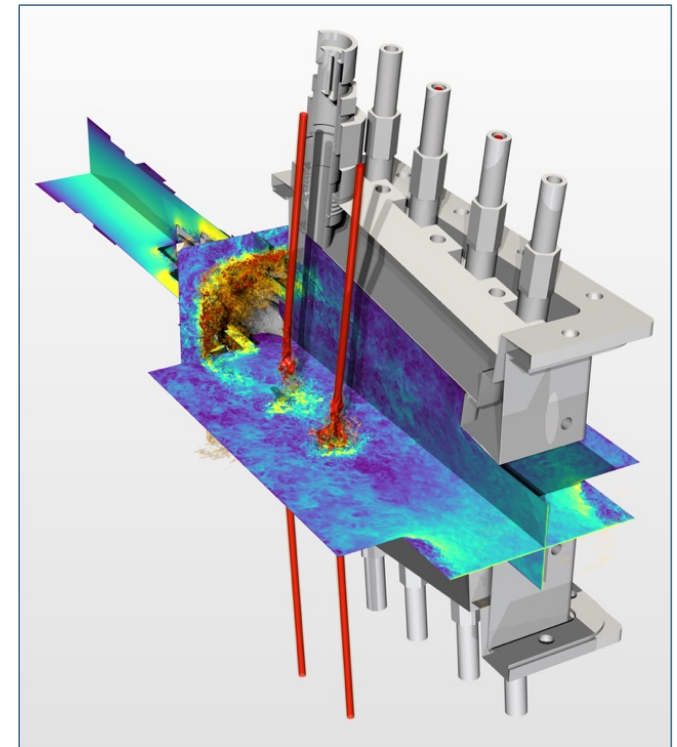
2015



2017

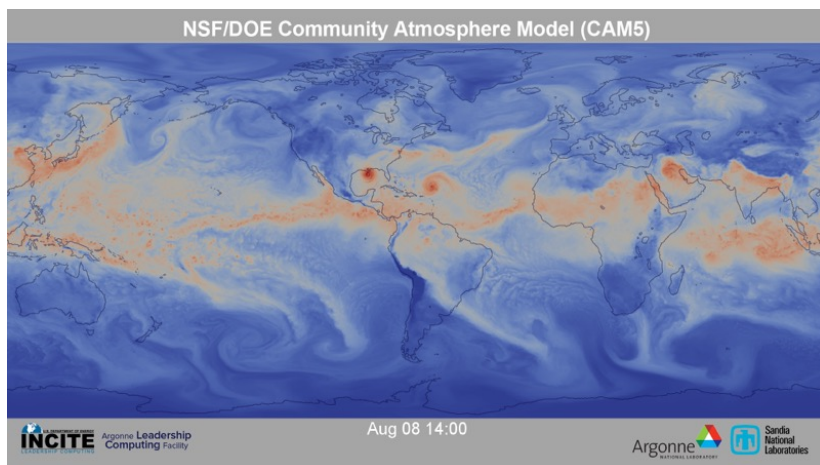


2021



2023

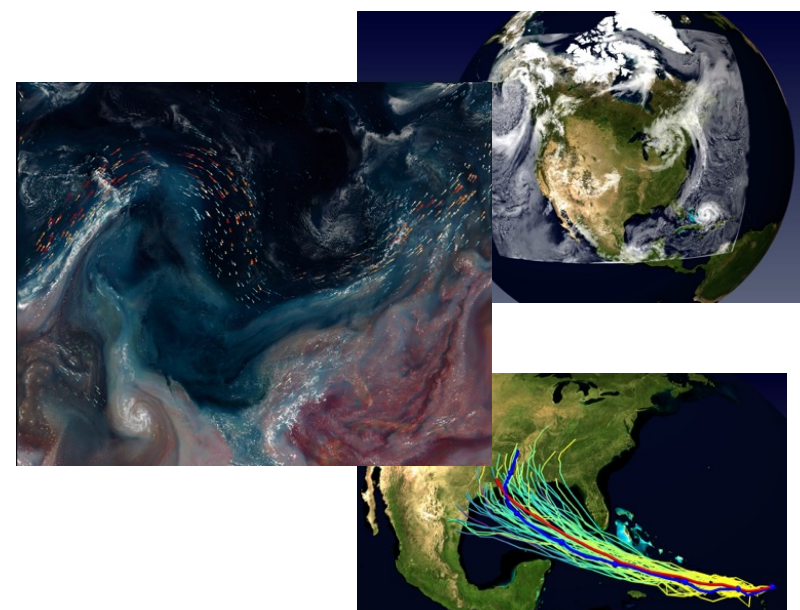
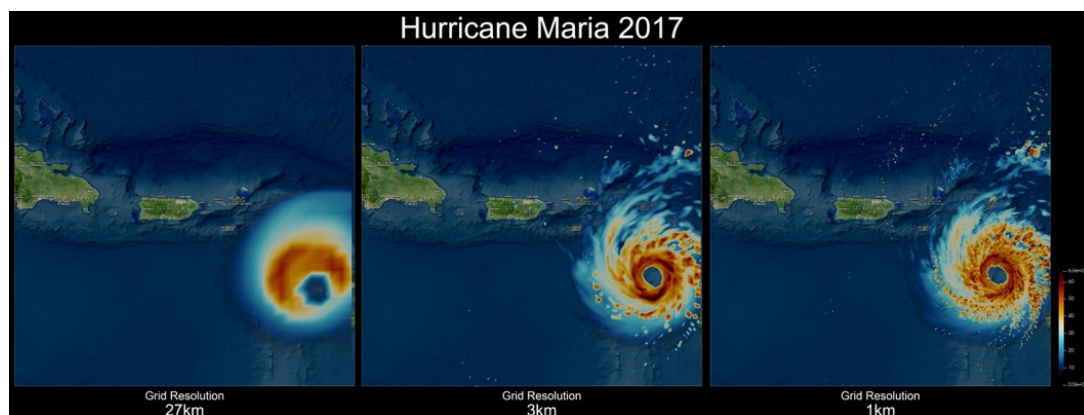
Earth Systems Science



PI: Warren Washington, National Center for Atmospheric Research

2012

2022



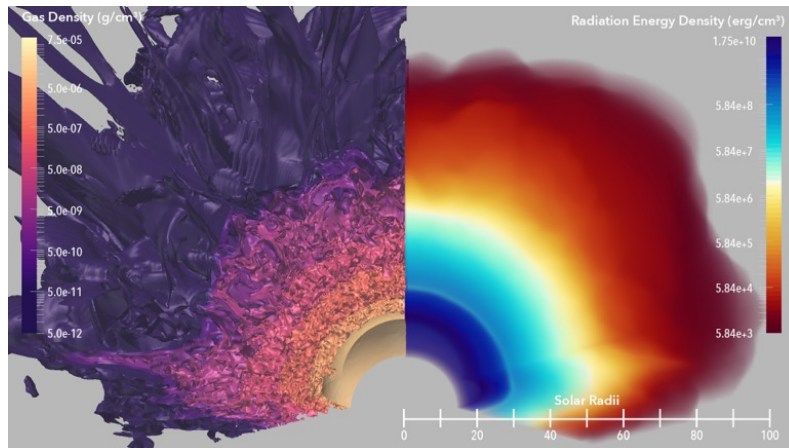
2024 / 2025

PI: Rao Kotamarthi, Argonne National Laboratory

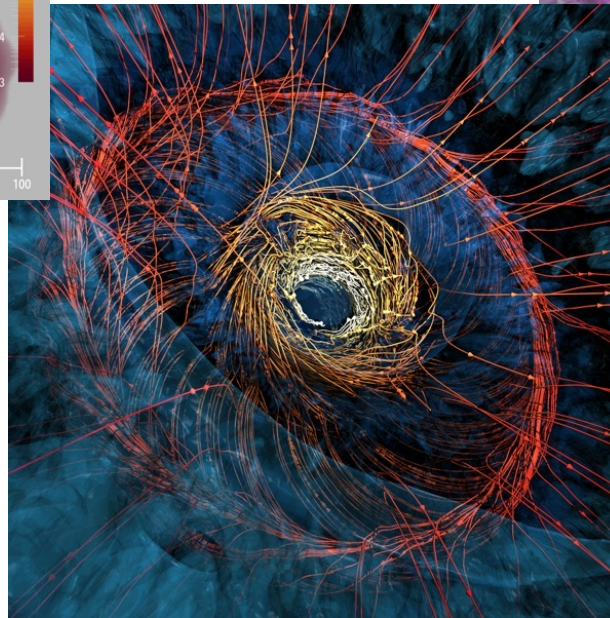
Physics: Stellar Radiation

2018

PI: Lars Bildsten, University of California, Santa Barbara



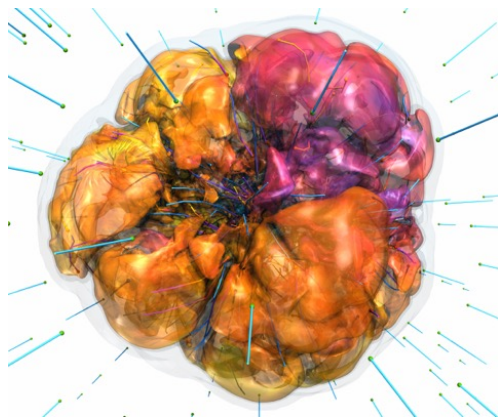
2017



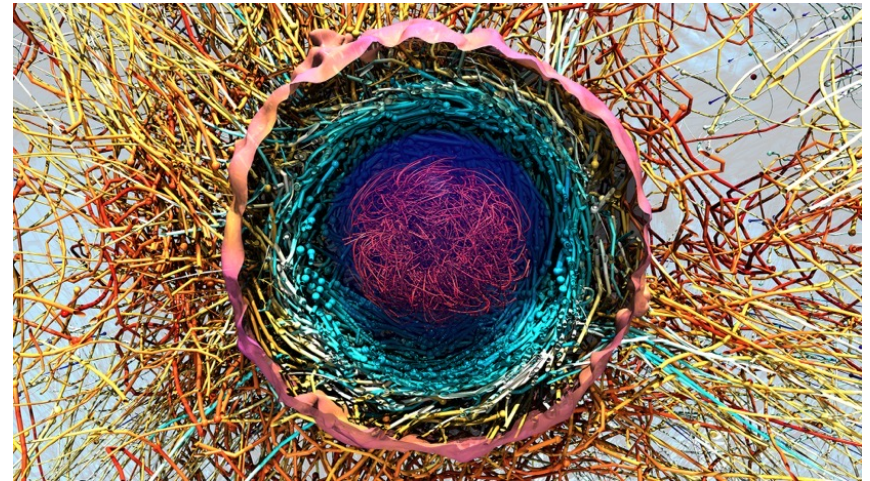
2021

Astrophysics

PI: Adam Burrows, Princeton University



2019

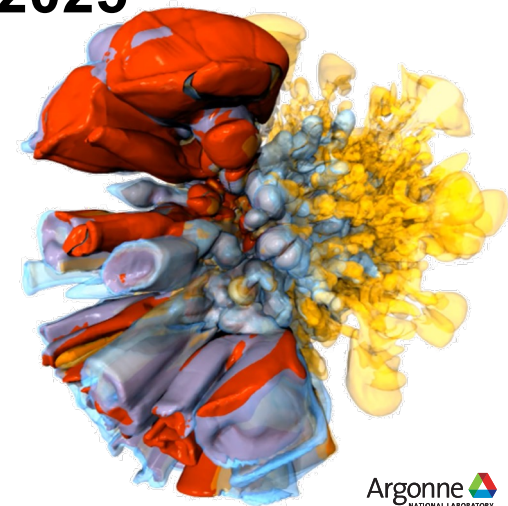
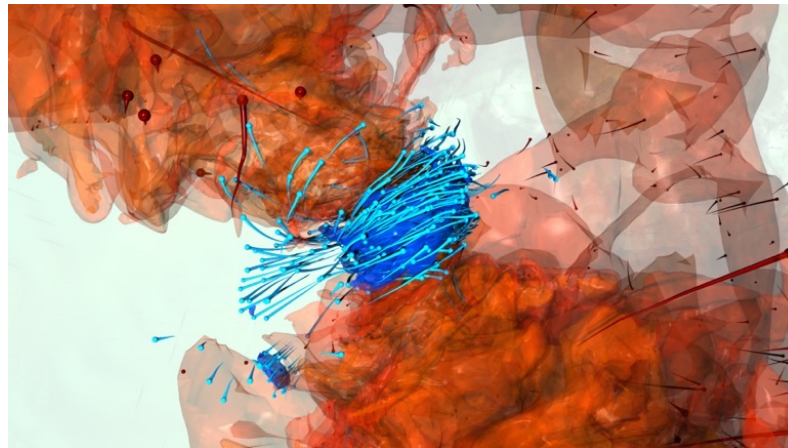
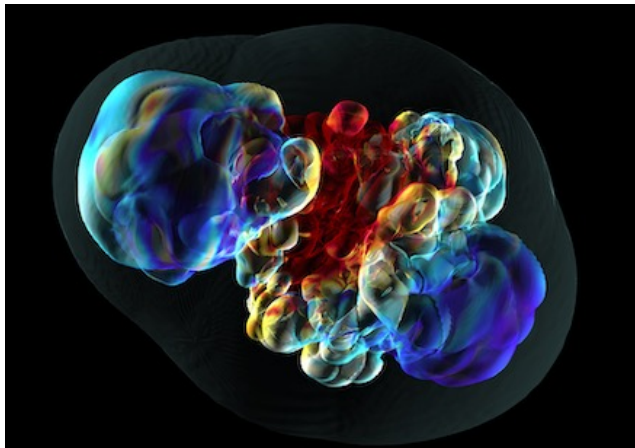


2021

2022

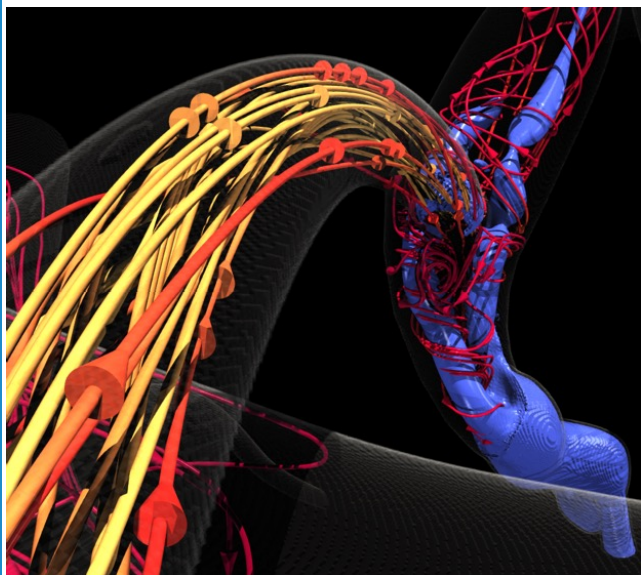
2023

2025

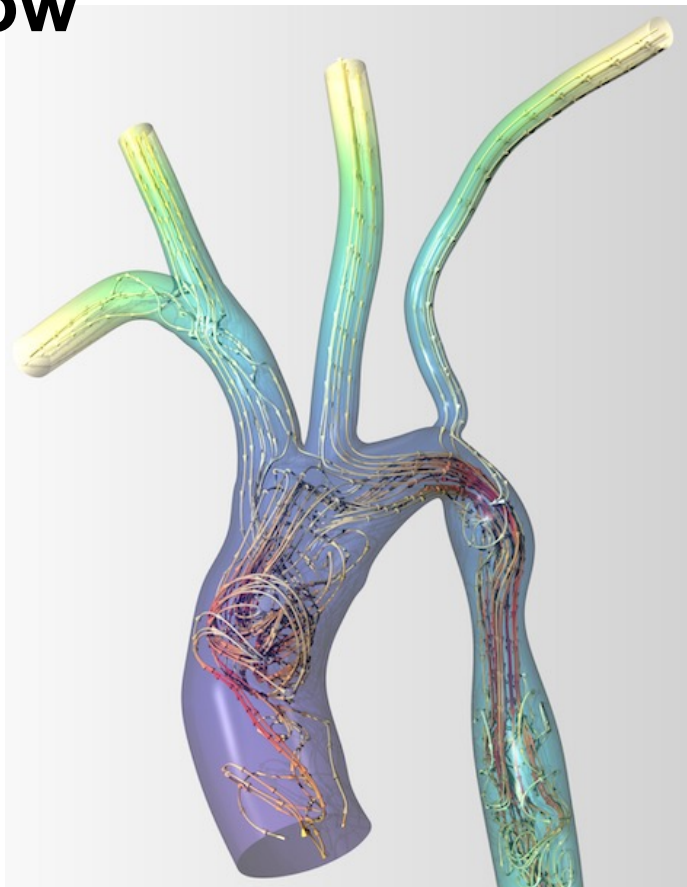


Arterial Blood Flow

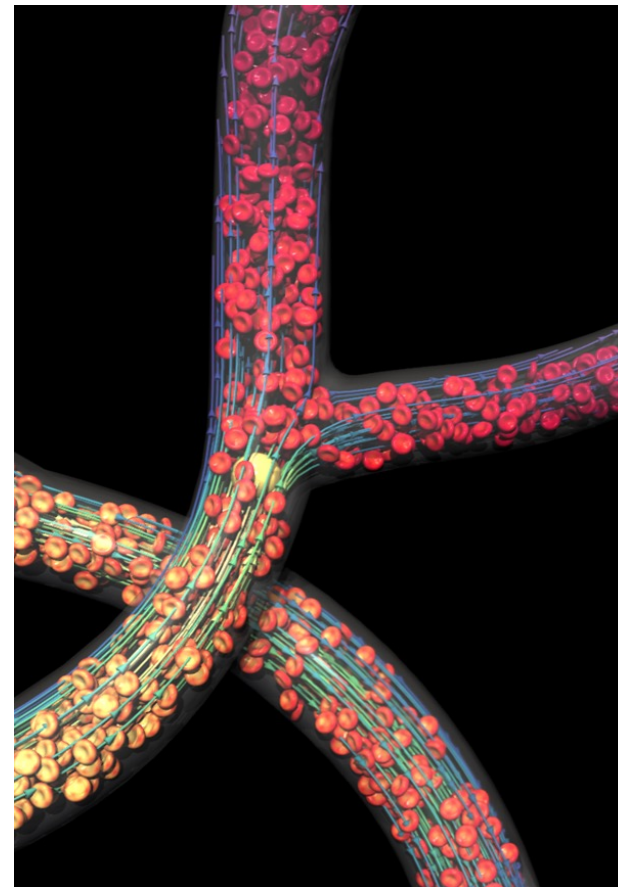
PI: Amanda Randles, Duke University



2020

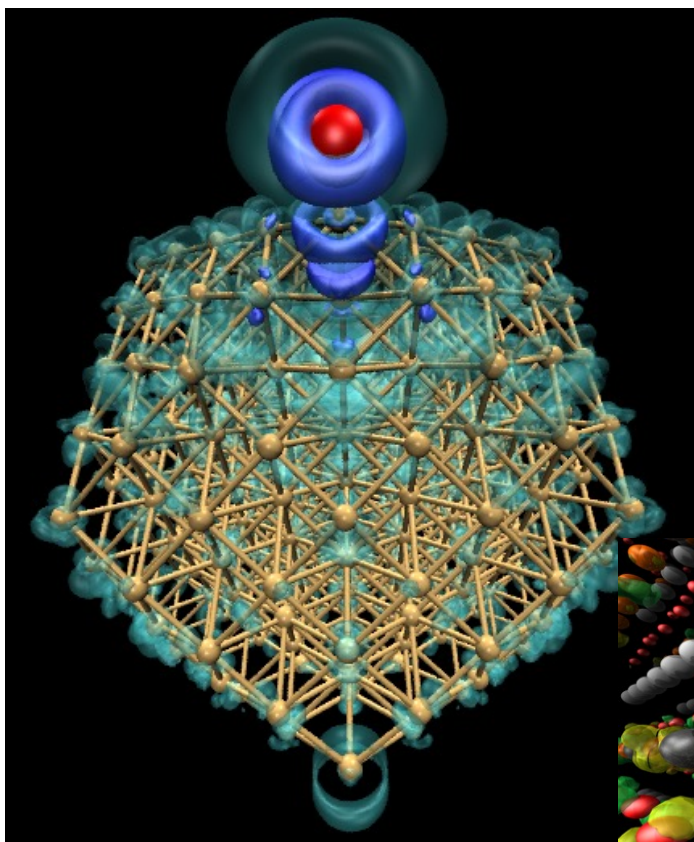


2023 Rendered on Aurora



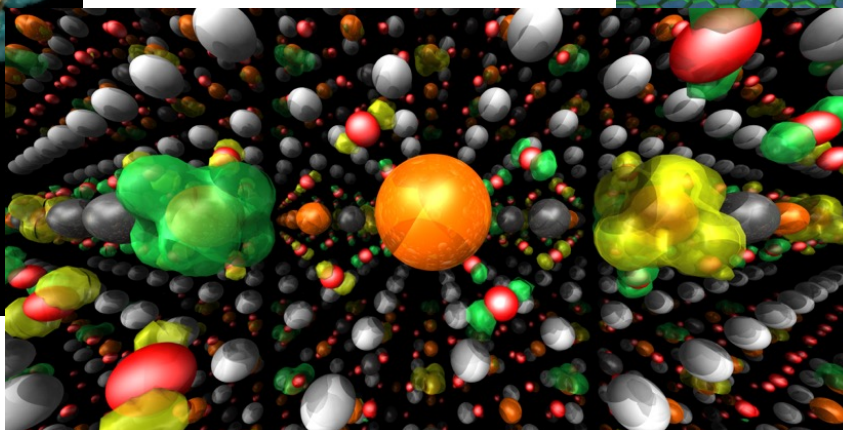
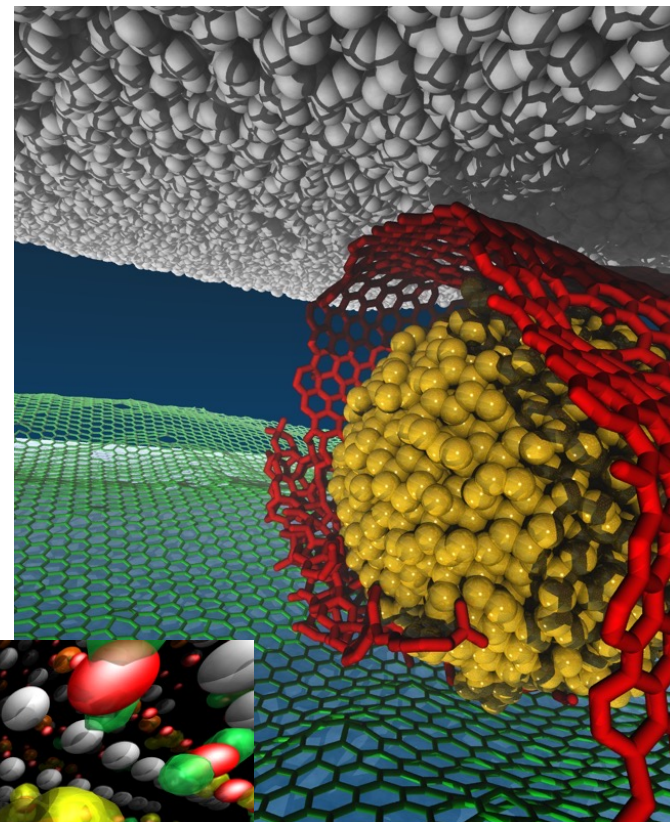
2023 Rendered on Aurora

Materials Science / Molecular



Data courtesy of: Jeff Greeley, Nichols Romero, Argonne National Laboratory

Data courtesy of:
Subramanian
Sankaranarayanan,
Argonne National
Laboratory



Data courtesy of: Paul Kent,
Oak Ridge National
Laboratory, Anouar Benali,
Argonne National Laboratory

Visualization Tools on Aurora and Polaris

Visualization Applications

- VisIt
- ParaView

Domain Specific

- VMD

APIs

- VTK: visualization

Utilities

- ImageMagick
- ffmpeg
- VNC

ParaView & VisIt vs. vtk

ParaView & VisIt

- General purpose visualization applications
- GUI-based
- Client / Server model to support remote visualization
- Scriptable / Extendable
- Built on top of vtk (largely)
- *In situ* capabilities



vtk

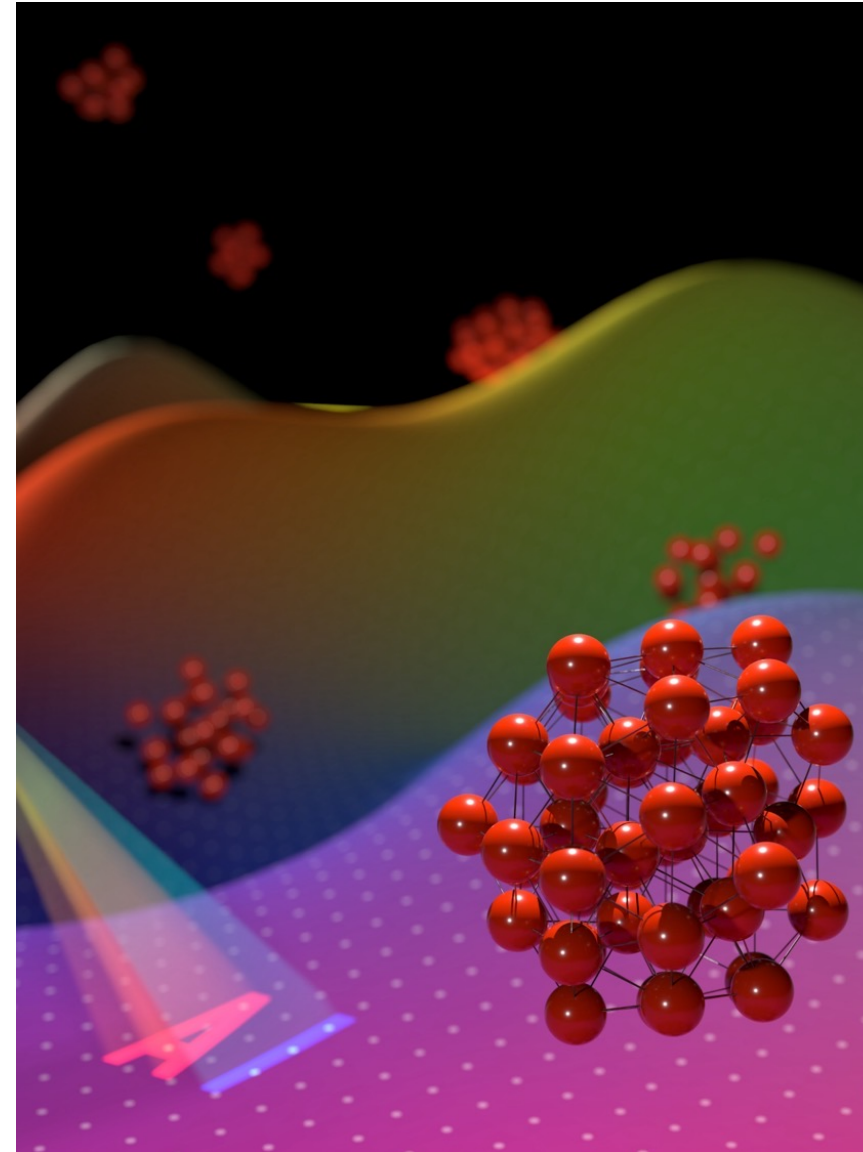
- Programming environment / API
- Additional capabilities, finer control
- Smaller memory footprint
- Requires more expertise (build custom applications)



Advanced Rendering

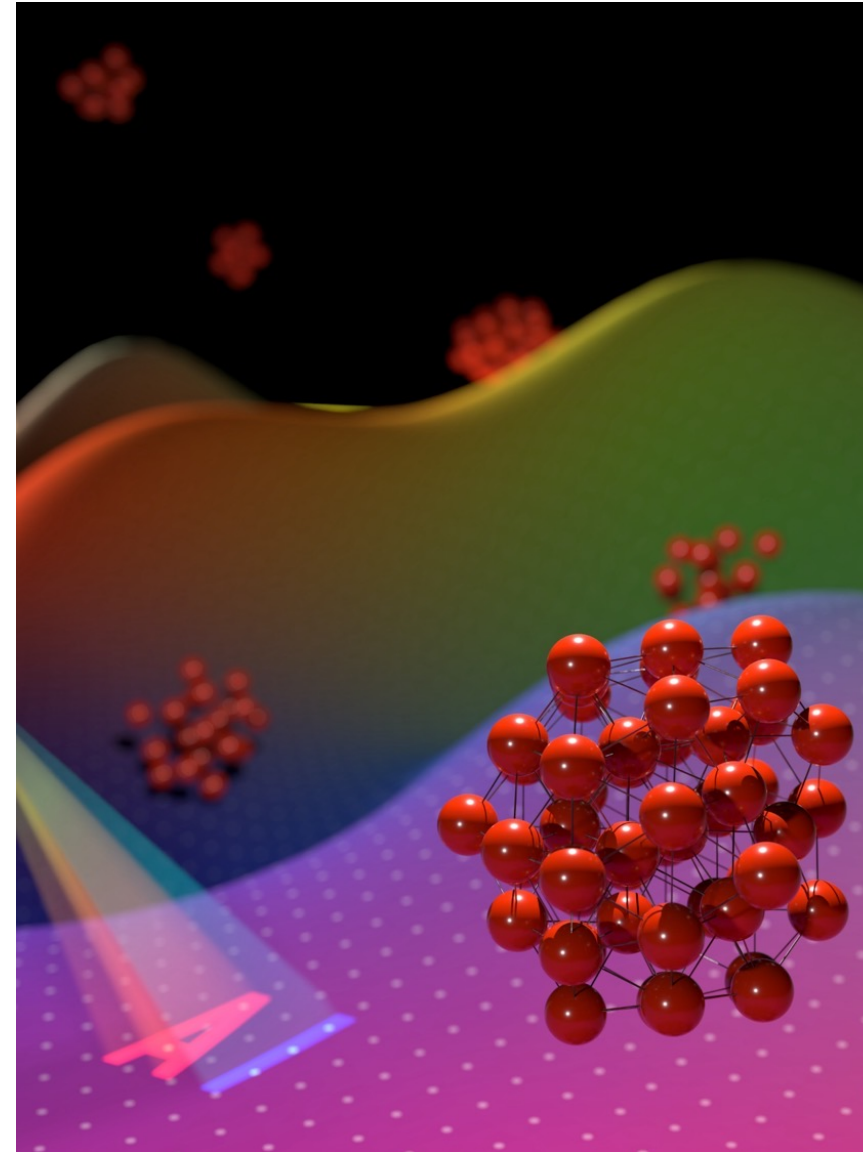
Why Advanced Rendering?

- Analysis → Communication
 - Tell a clearer story
 - Hold attention
 - Communicate to non-experts
- Clearer communication
 - Shadows separate structures
 - Materials distinguish regions
 - Maintains spatial relationships
- Overall: Less cognitive load for the viewer



Why Advanced Rendering?

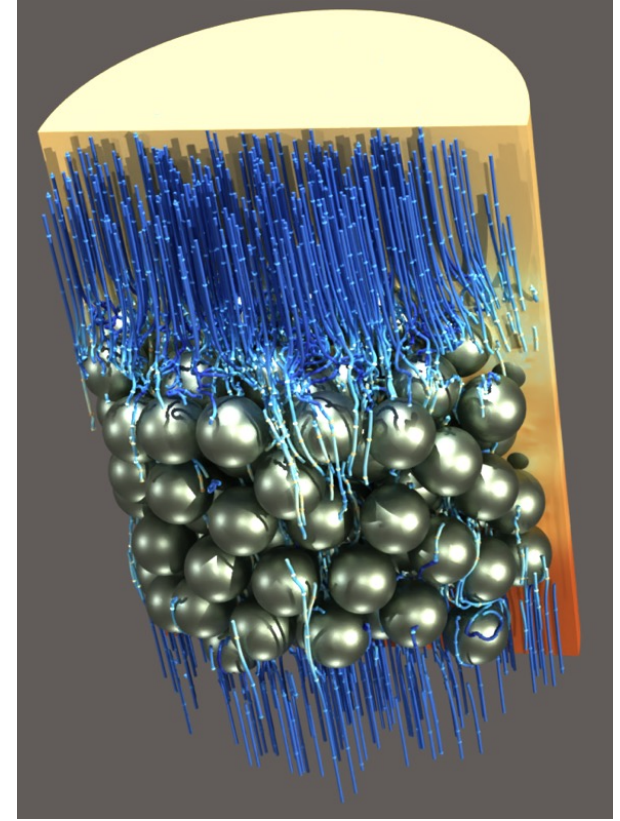
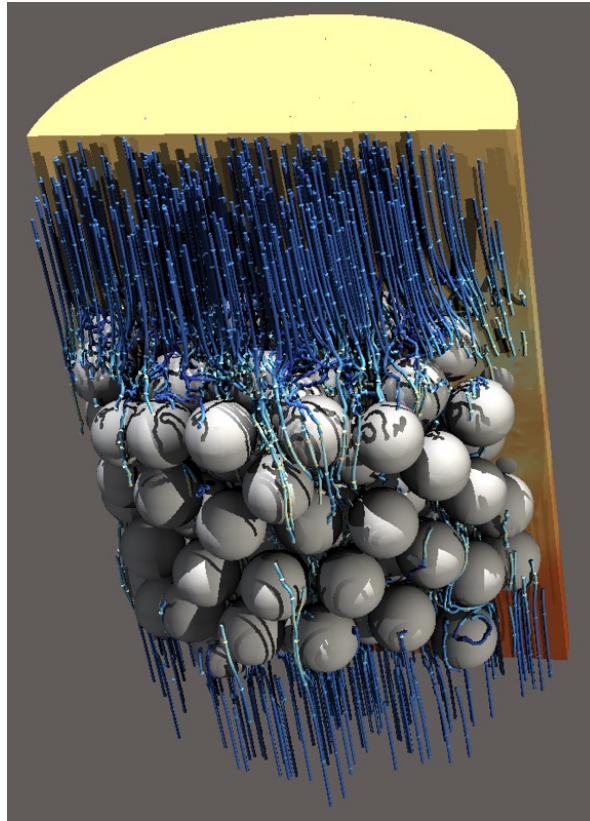
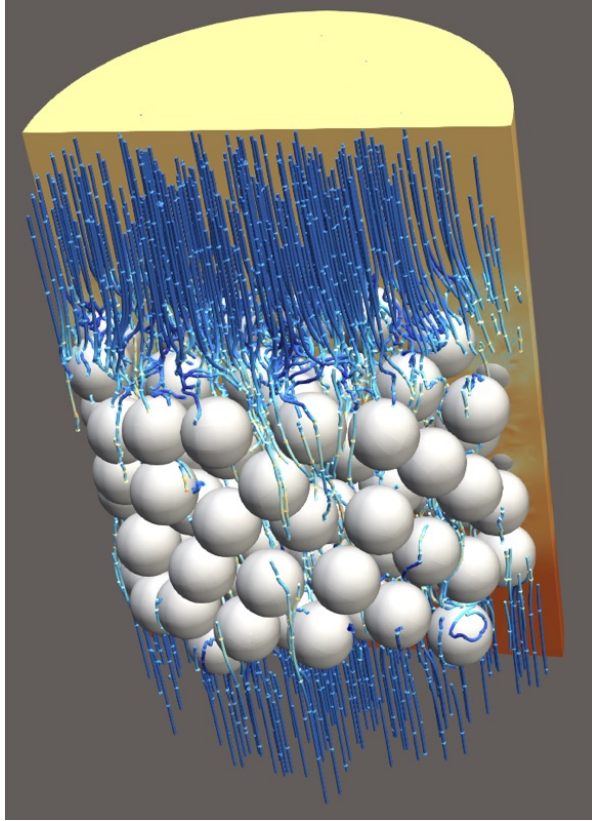
- Rendering mode
 - Rasterization / raytracing / pathtracing
- Materials
- Lighting



Rasterization / Ray Tracing / Path Tracing

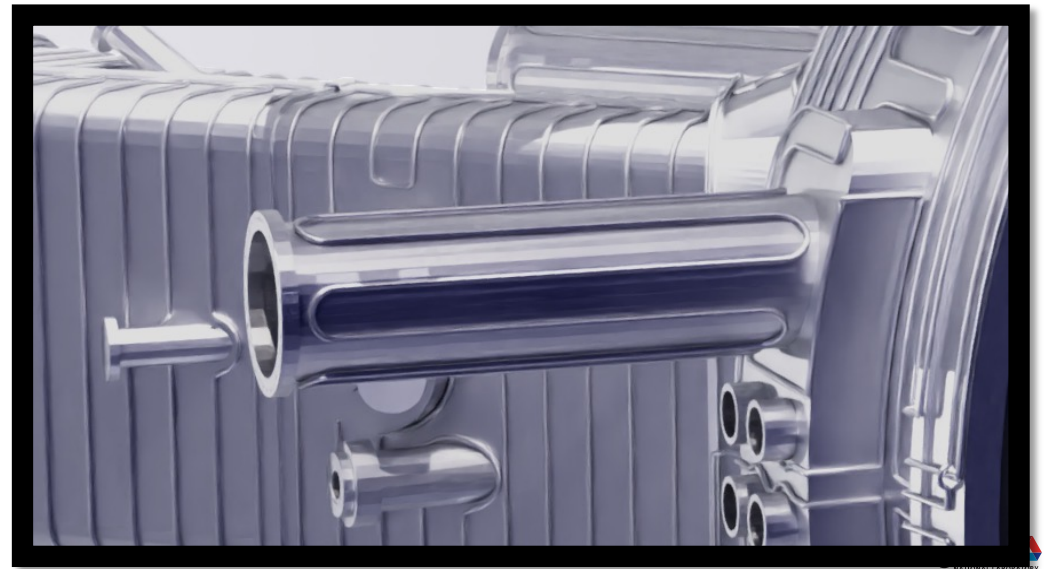
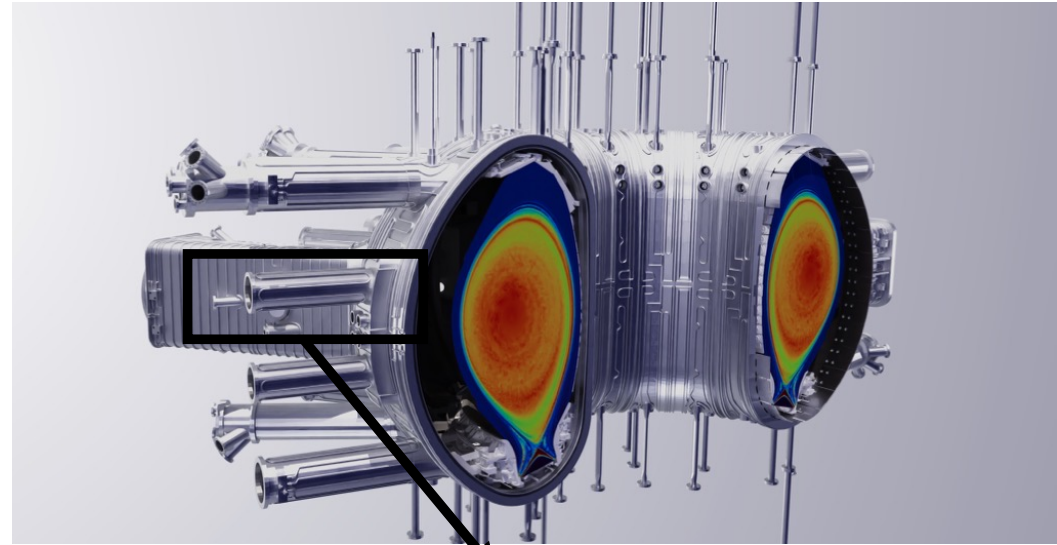
- Rasterization
 - Projects geometry onto the screen
- Ray Tracing
 - Shoots rays from the camera into the scene
- Path Tracing
 - Extends ray tracing with many random light paths
 - Physically correct, but expensive
- Tradeoff: Realism vs Compute cost

Rasterization / Ray Tracing / Path Tracing



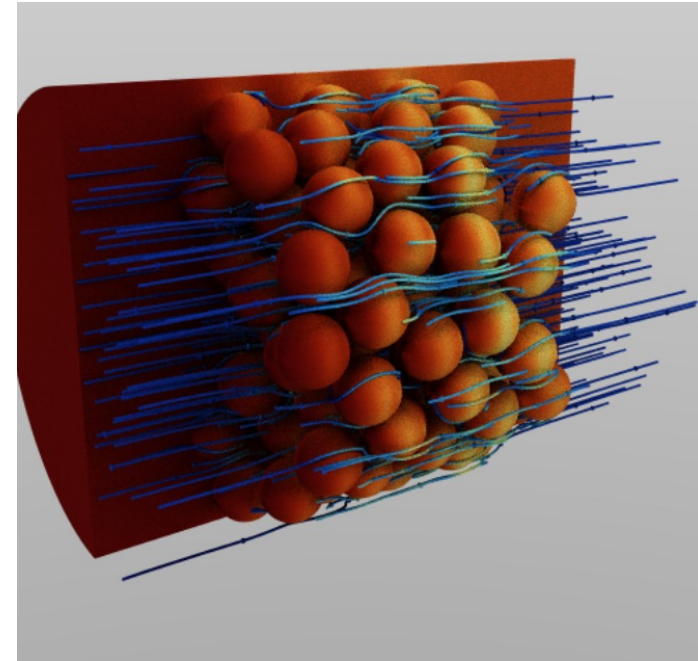
Materials

- Communicate what the object is
—Wood, plastic, glass, ceramic
- Provides context and makes the image/video more realistic
- Declutters the image



Lighting

- Why use lights?
 - Lighting is 80% of the image
 - Shadows create shape, help with depth perception
 - Provide instant realism
- Types
 - Directional, Positional



Pebble Bed Reactor simulation

ParaView

If you want to follow/replicate, data is here:

`/grand/projects/alcf_training/visualization/PEBBLE_BED_REACTOR_DATA`

PVSM state is here: `/grand/projects/alcf_training/visualization/PEBBLE_BED_REACTOR_PVSM`

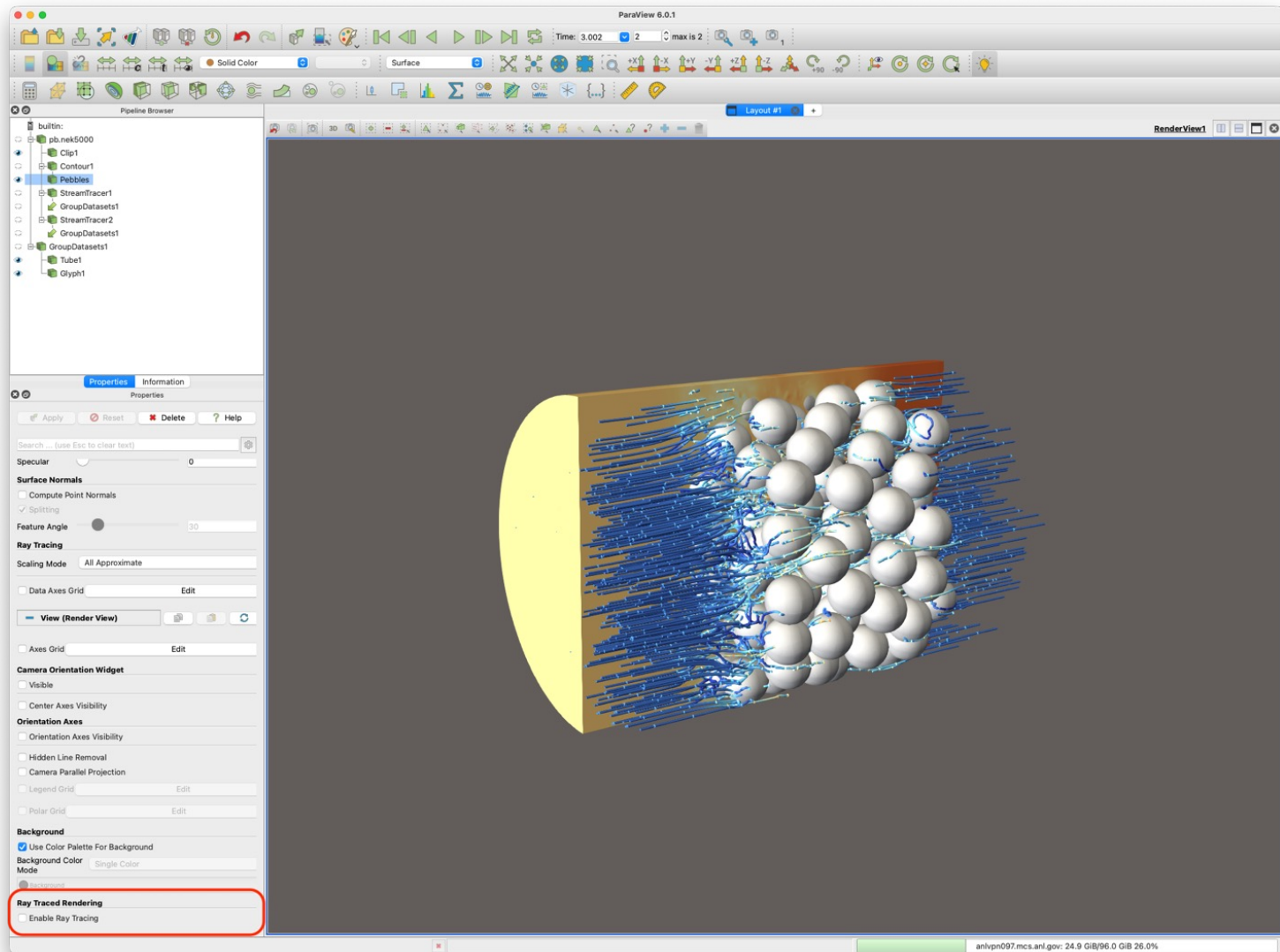
I have also put all rendered pngs

here: `/grand/projects/alcf_training/visualization/PEBBLE_BED_REACTOR_IMAGES`

And have embedded the PV state File. You can load the specific visualization by loading **File->Load State** and selecting the image.

You will need to change the path to the data.

ParaView



ParaView

Ray Traced Rendering

☒ Enable Ray Tracing

Back End OSPRay raycaster

☒ Shadows

Ambient Samples

Samples Per Pixel

Progressive Passes

Roulette Depth

☒ Denoise

Light Scale

Temporal Cache Size

Background mode Environment

Environment North

Environment East

Path Traced Environment

Single color

☒ Color Restore Default

ParaView

Ray Traced Rendering

☒ Enable Ray Tracing

Back End OSPRay raycaster

☒ Shadows

Ambient Samples

☒ Samples Per Pixel

Progressive Passes

Roulette Depth

☒ Denoise

Light Scale

Temporal Cache Size

Background mode Environment

Environment North

Environment East

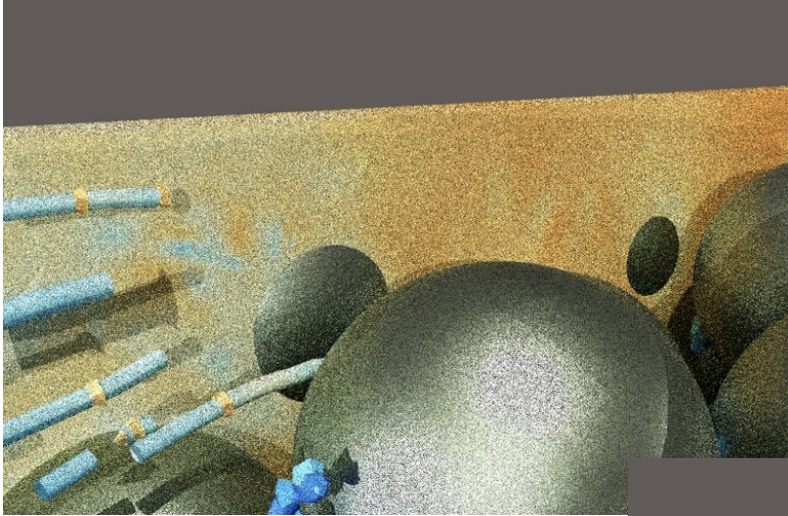
Path Traced Environment

Single color

☐ Color

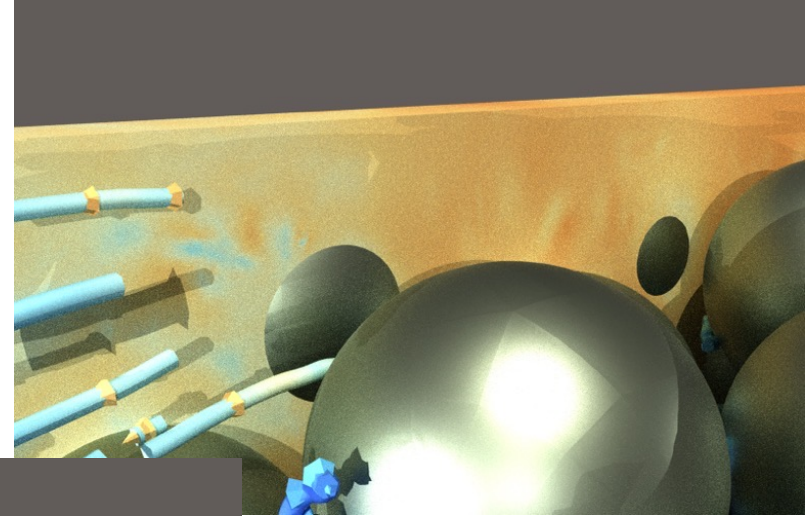
Restore Default

Samples per Pixel (pathtraced)

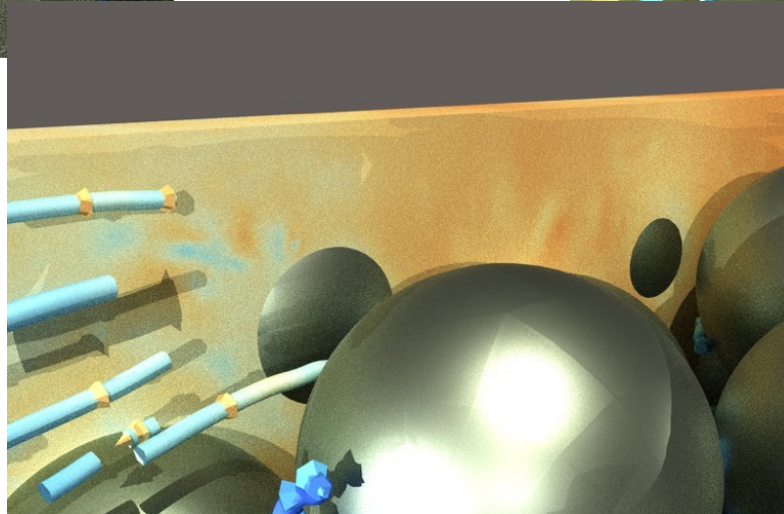


1 Sample per Pixel

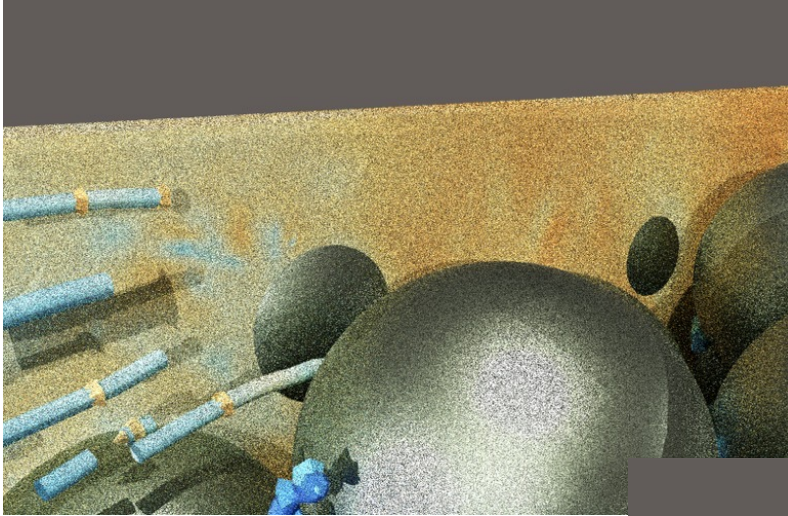
8 Samples per Pixel



16 Samples per Pixel

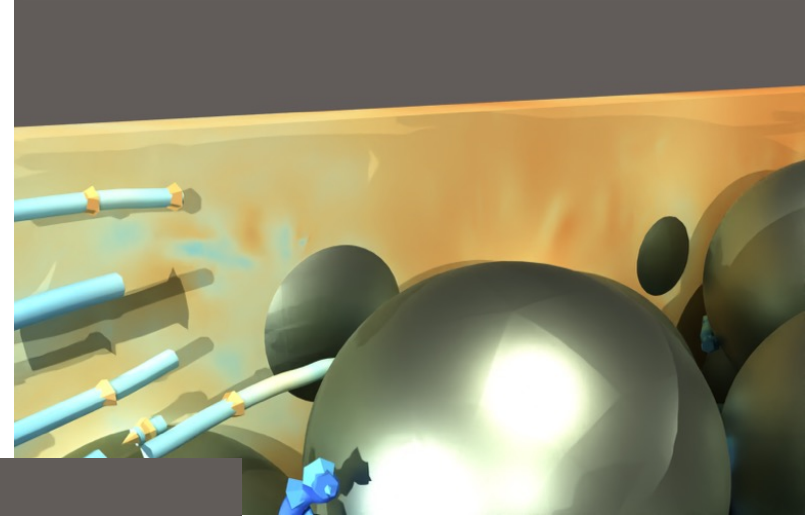


Samples per Pixel (pathtraced + denoised)

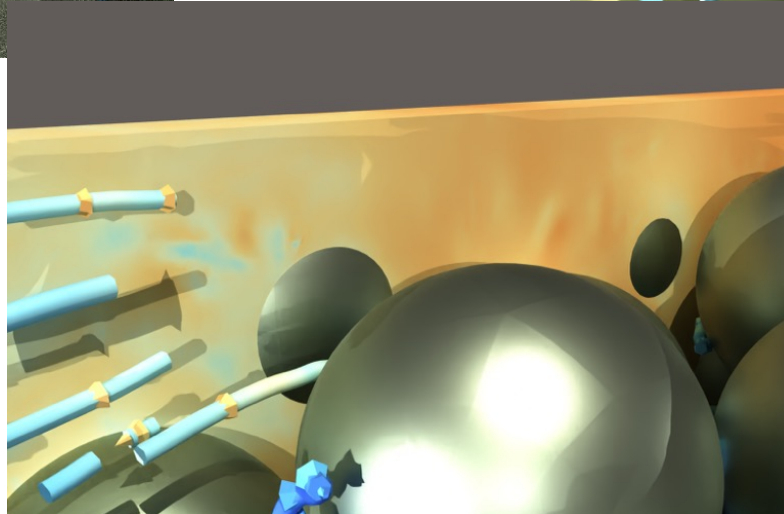


1 Sample per Pixel

8 Samples per Pixel



16 Samples per Pixel



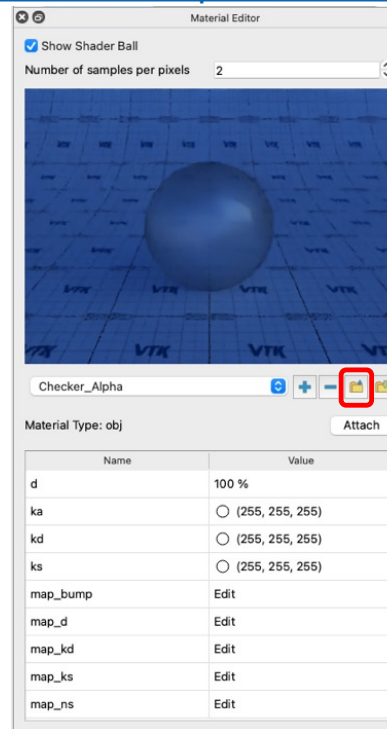
! *You can use denoise only
with 4 pixels per sample and
above*

Materials

! *Materials do not ship with ParaView. You need to download and install them separately*

Download Materials here: <https://gitlab.kitware.com/paraview/materials>

View -> Material Editor



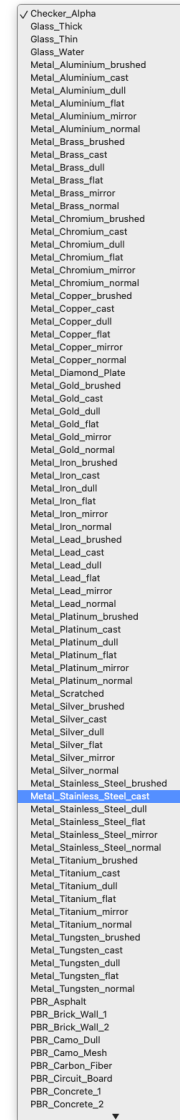
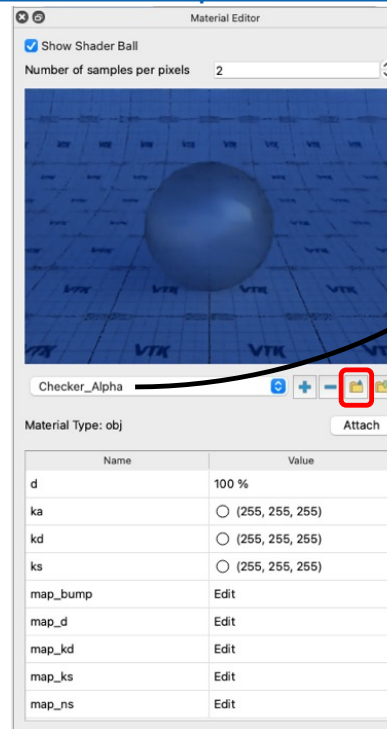
! *Materials only work with pathtracer*

Materials

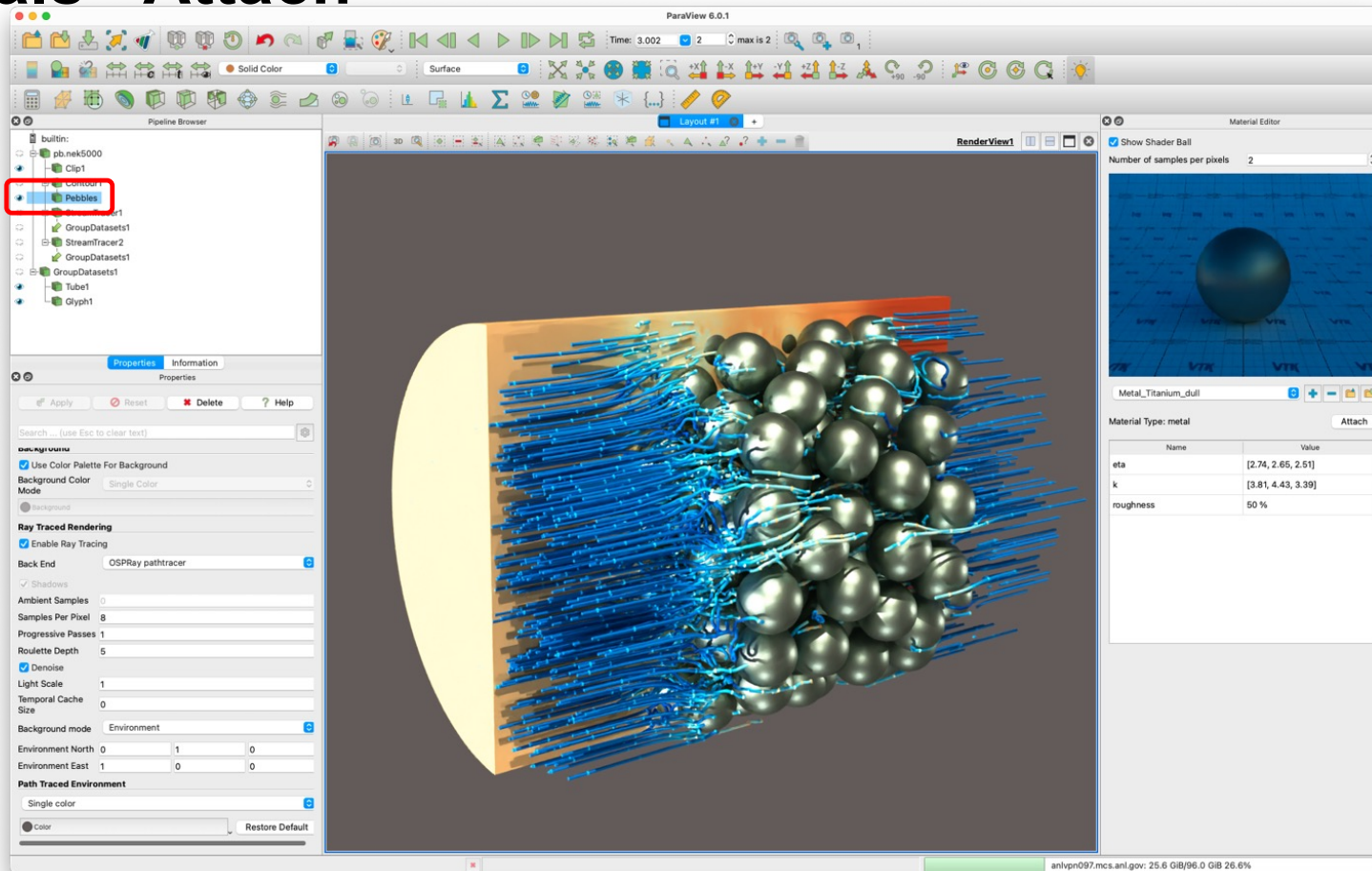
! *Materials do not ship with ParaView. You need to download and install them separately*

Download Materials here: <https://gitlab.kitware.com/paraview/materials>

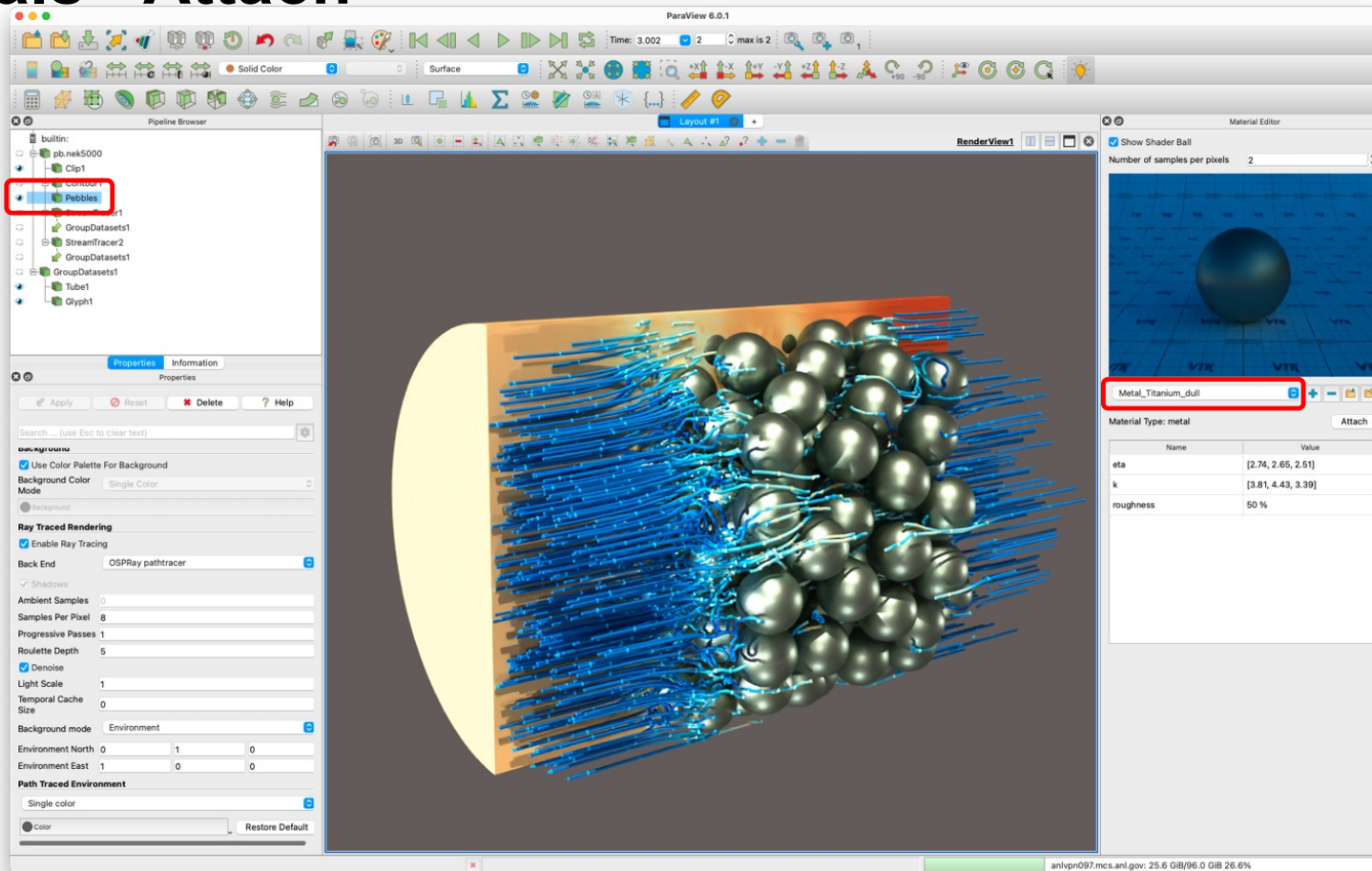
View -> Material Editor



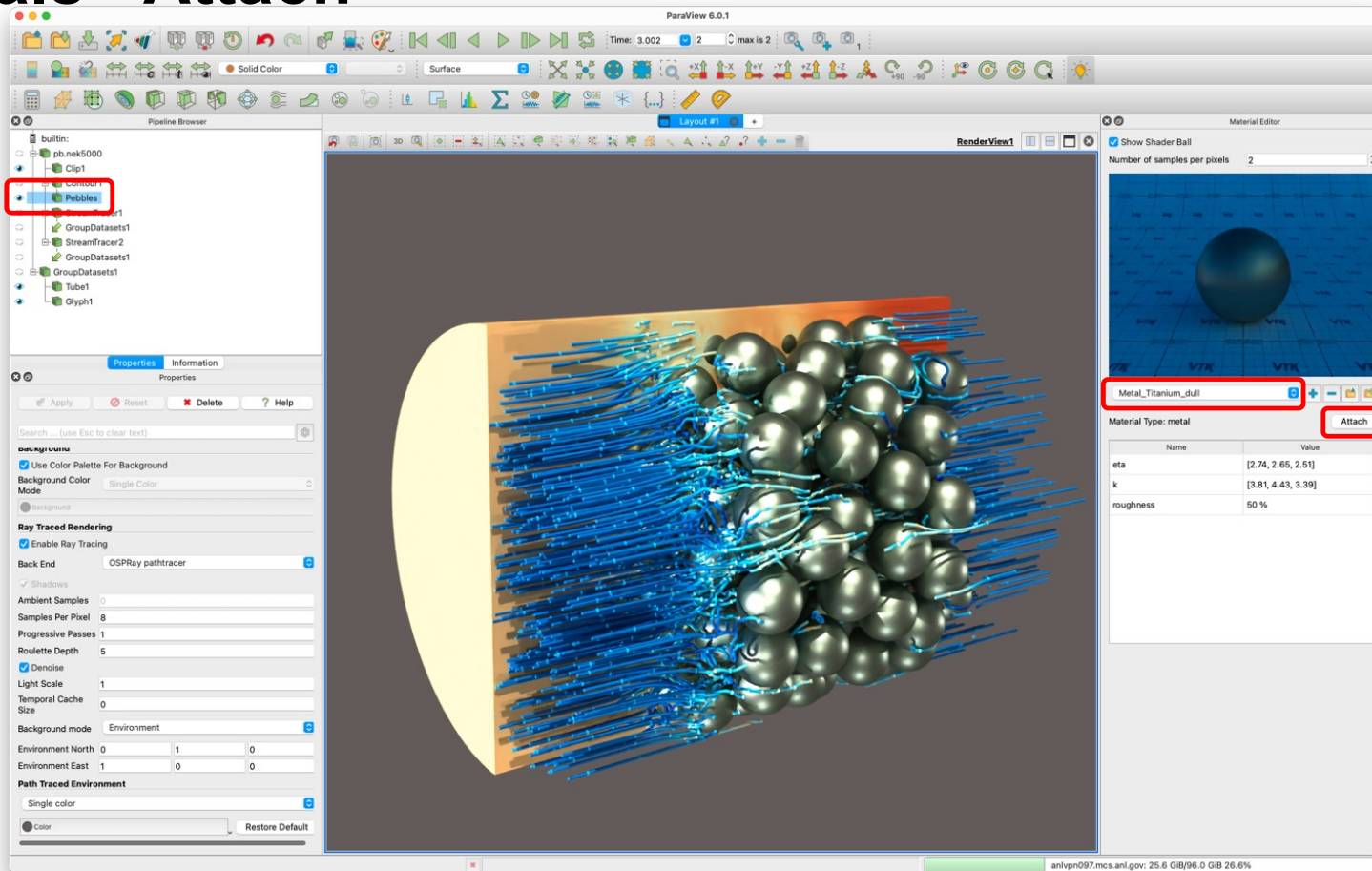
Materials - Attach



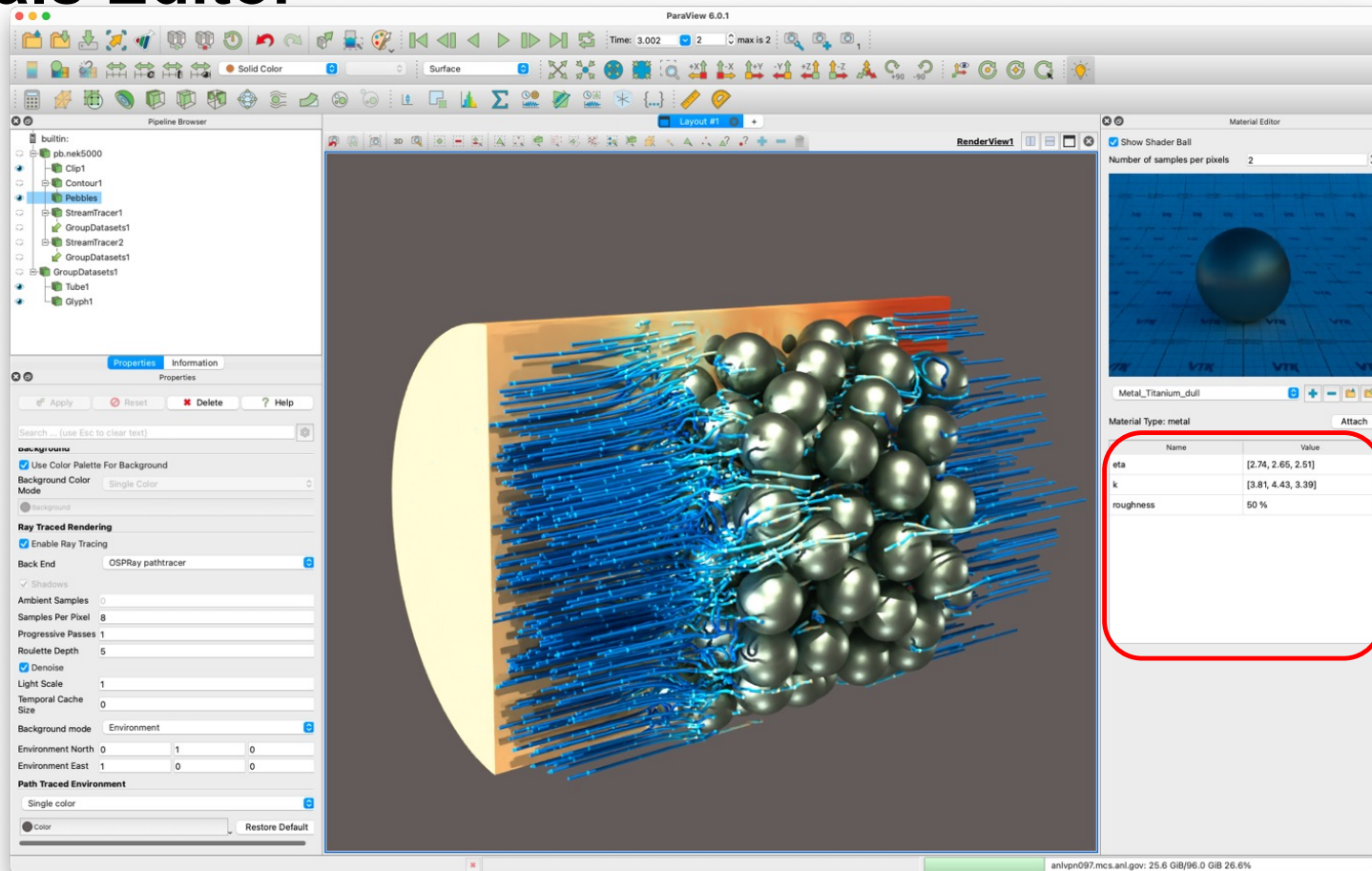
Materials - Attach



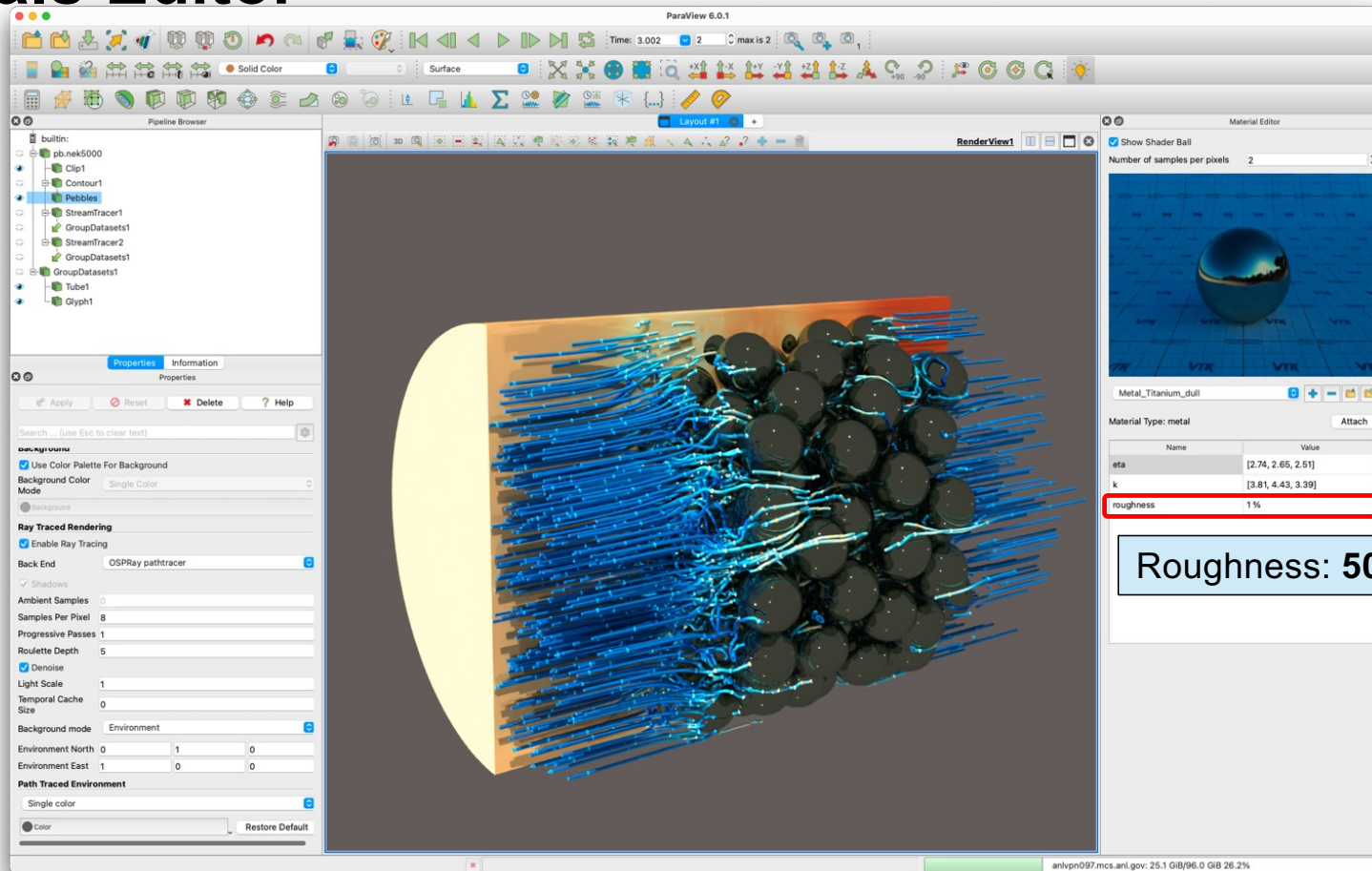
Materials - Attach



Materials Editor



Materials Editor

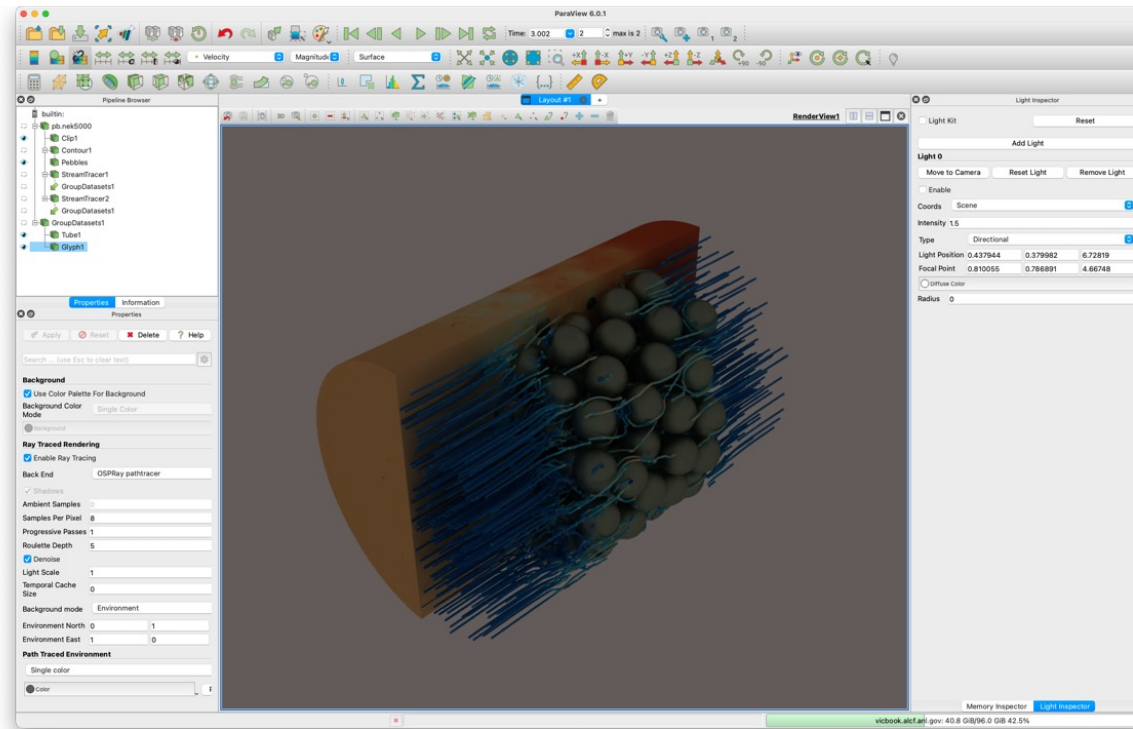


Roughness: 50% -> 1%

ParaView -> Lights

Why are lights important?

- **Drives realism:** Light defines how materials look (reflections, roughness, etc.)
- **Reveals form:** Shadows and highlights make shape and depth readable
- **Guides focus:** Directs attention to what matters in the scene

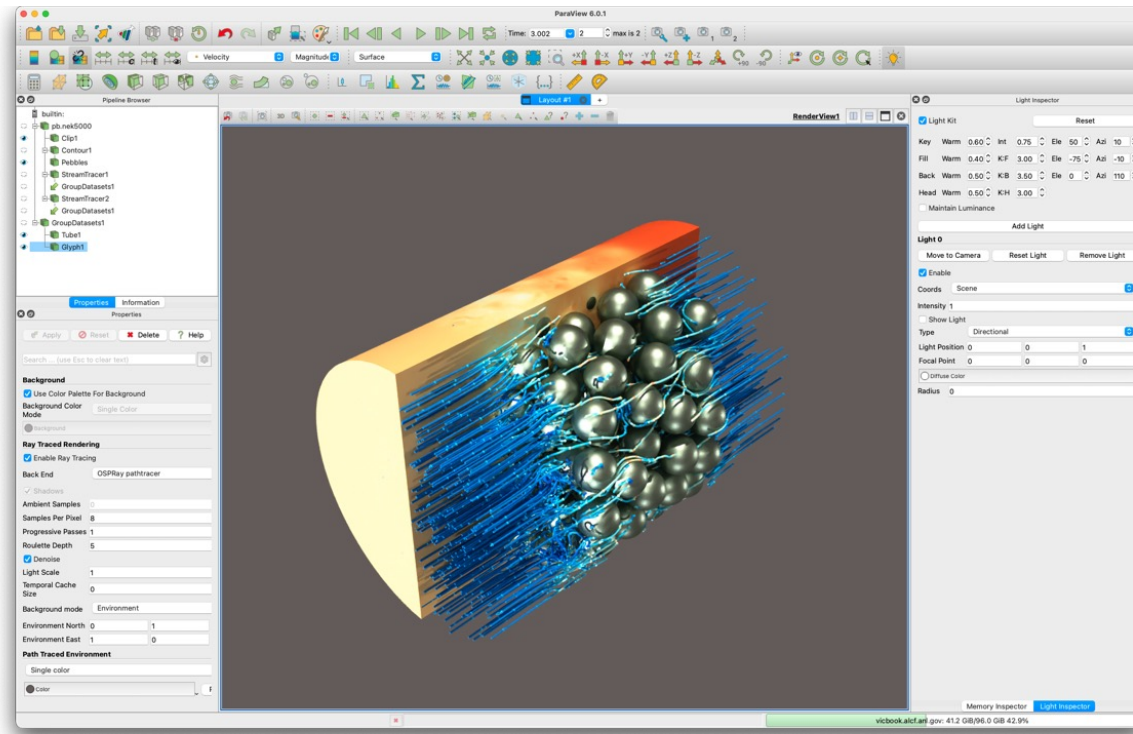


Scene with no lights

ParaView -> Lights

Why are lights important?

- **Drives realism:** Light defines how materials look (reflections, roughness, etc.)
- **Reveals form:** Shadows and highlights make shape and depth readable
- **Guides focus:** Directs attention to what matters in the scene



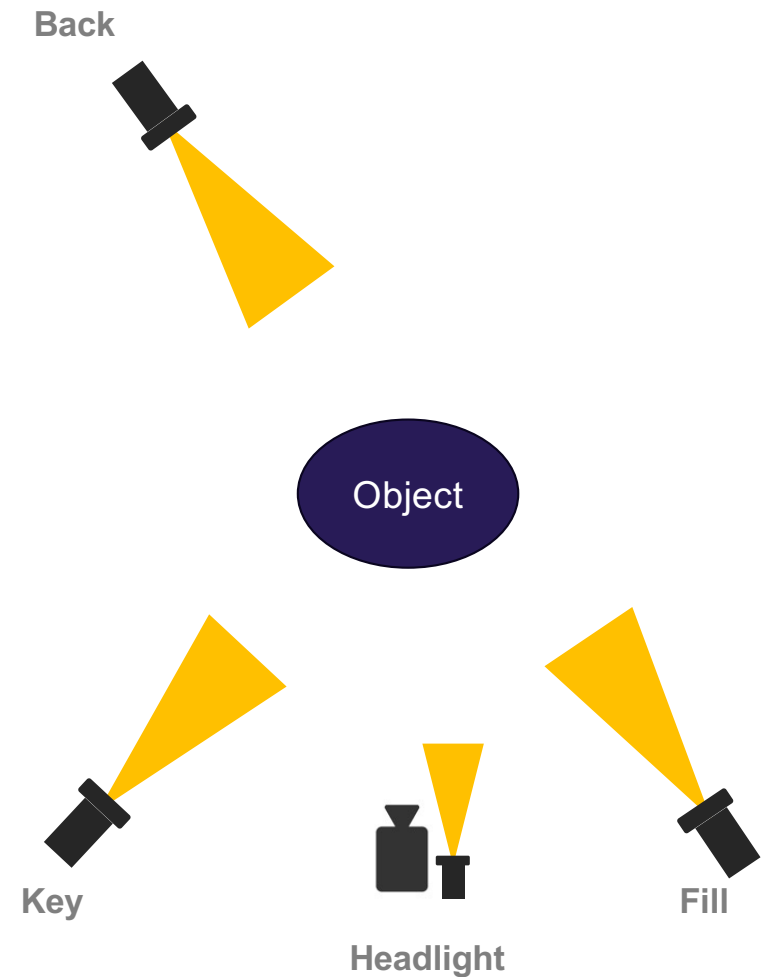
Scene with lights

Lighting – 3-point lighting

- **Key Light:** The primary light source, typically bright and overhead.
- **Fill Light:** Positioned opposite the key light to reduce harsh shadows.
- **Back Lights:** Two weaker lights from behind to improve edge definition.
- **Headlight:** Aligned with the camera to reduce contrast and ensure visibility.

Types

- **Directional:** Light comes from an infinite distance with parallel rays (like sunlight), only direction matters.
- **Positional:** Light originates from a specific point in space and spreads outward; both position and direction affect the scene.



ParaView -> Lights

View -> Light Inspector

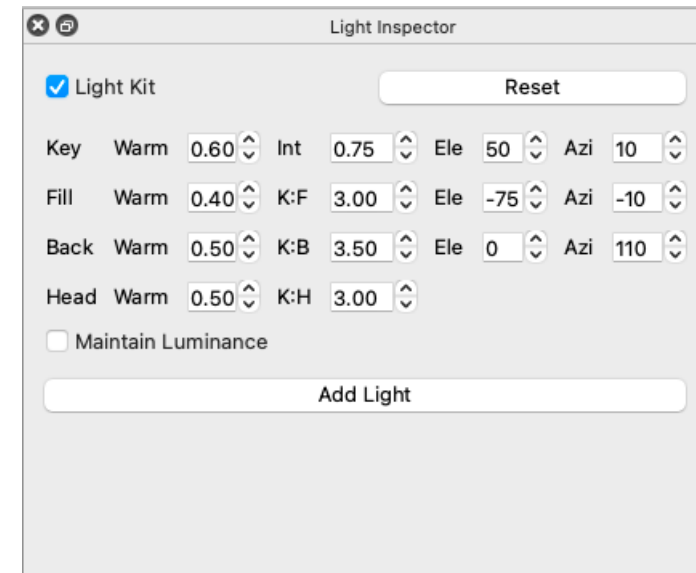
- Warm: Color of the light along a warmth scale from 0 (cold blue) to 1 (warm red). A value of 0.5 is neutral white.
- Int: The intensity of the key light (0 = off, 1 = full intensity).
- K:F, K:B, K:H: Ratios for key-to-fill, key-to-back, and key-to-headlight intensities.

Recommended ranges are:

- K:F: 2-10, K:B: 2-10, K:H: 2-15
- Ele (Elevation) and Azi (Azimuth): Define the vertical and horizontal angles of each light relative to the camera's lookat point, defined in degrees.

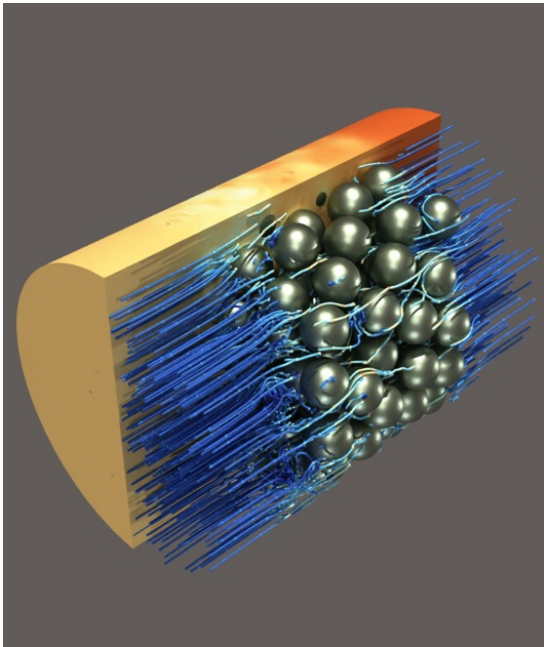
For example:

- (Elevation=90, Azimuth=0): directly above the scene.
- (Elevation=0, Azimuth=0): at the camera (i.e., a headlight).
- (Elevation=45, Azimuth=-20): above and in front, slightly to the left.

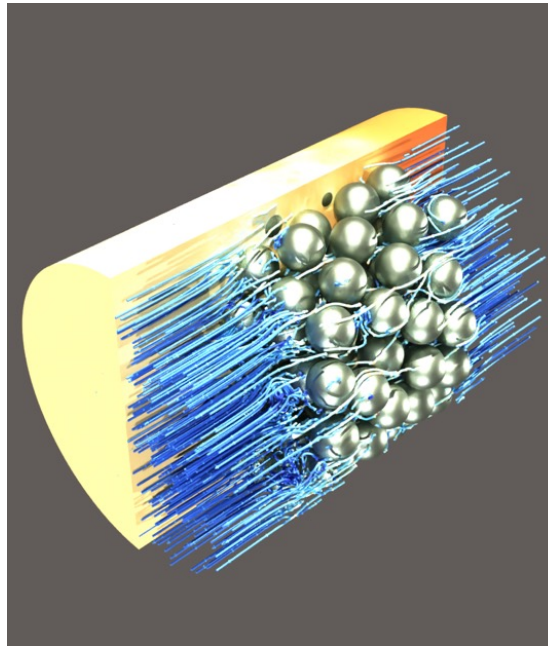


! *Light kit moves with the camera*

ParaView - Light Scale



Light Scale: 1



Light scale: 3

Ray Traced Rendering

☒ Enable Ray Tracing

Back End: OSPRay pathtracer

☒ Shadows

Ambient Samples: 0

Samples Per Pixel: 8

Progressive Passes: 1

Roulette Depth: 5

☒ Denoise

Light Scale: 1

Temporal Cache Size: 0

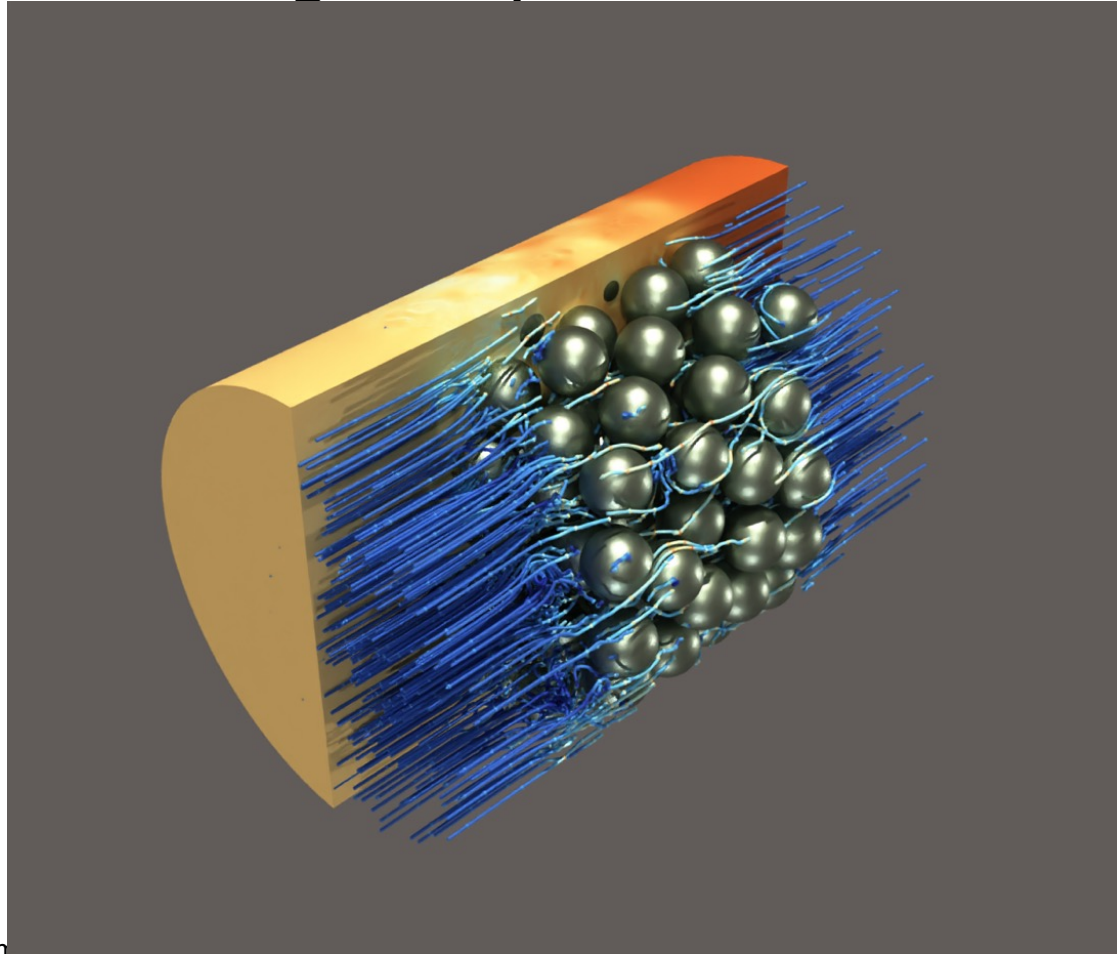
Background mode: Environment

Environment North: 0 1 0

Environment East: 1 0 0

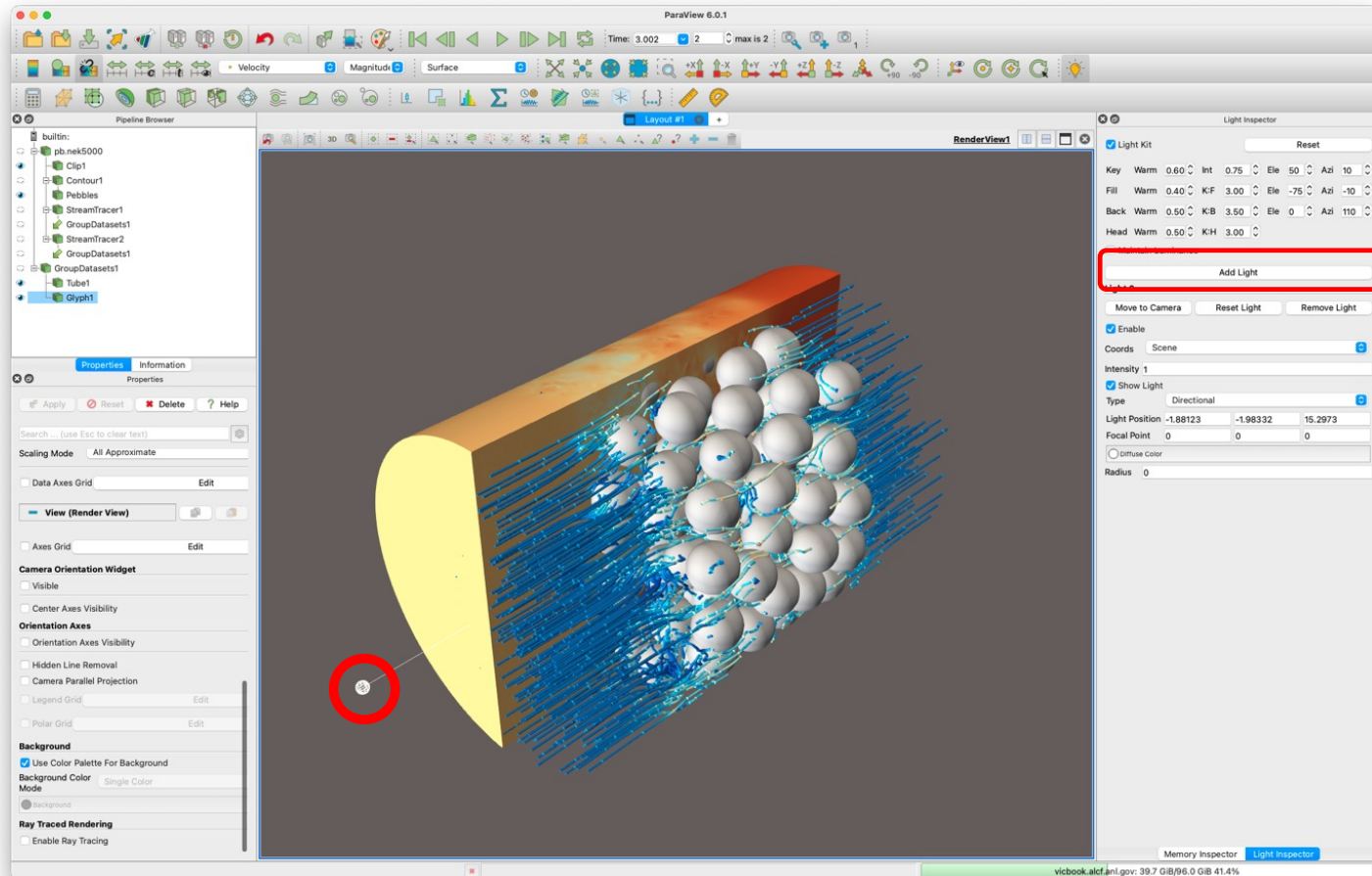
Sets the number of occlusion query rays that OSPRay raycaster sends at each intersection.

ParaView (default Light Kit)

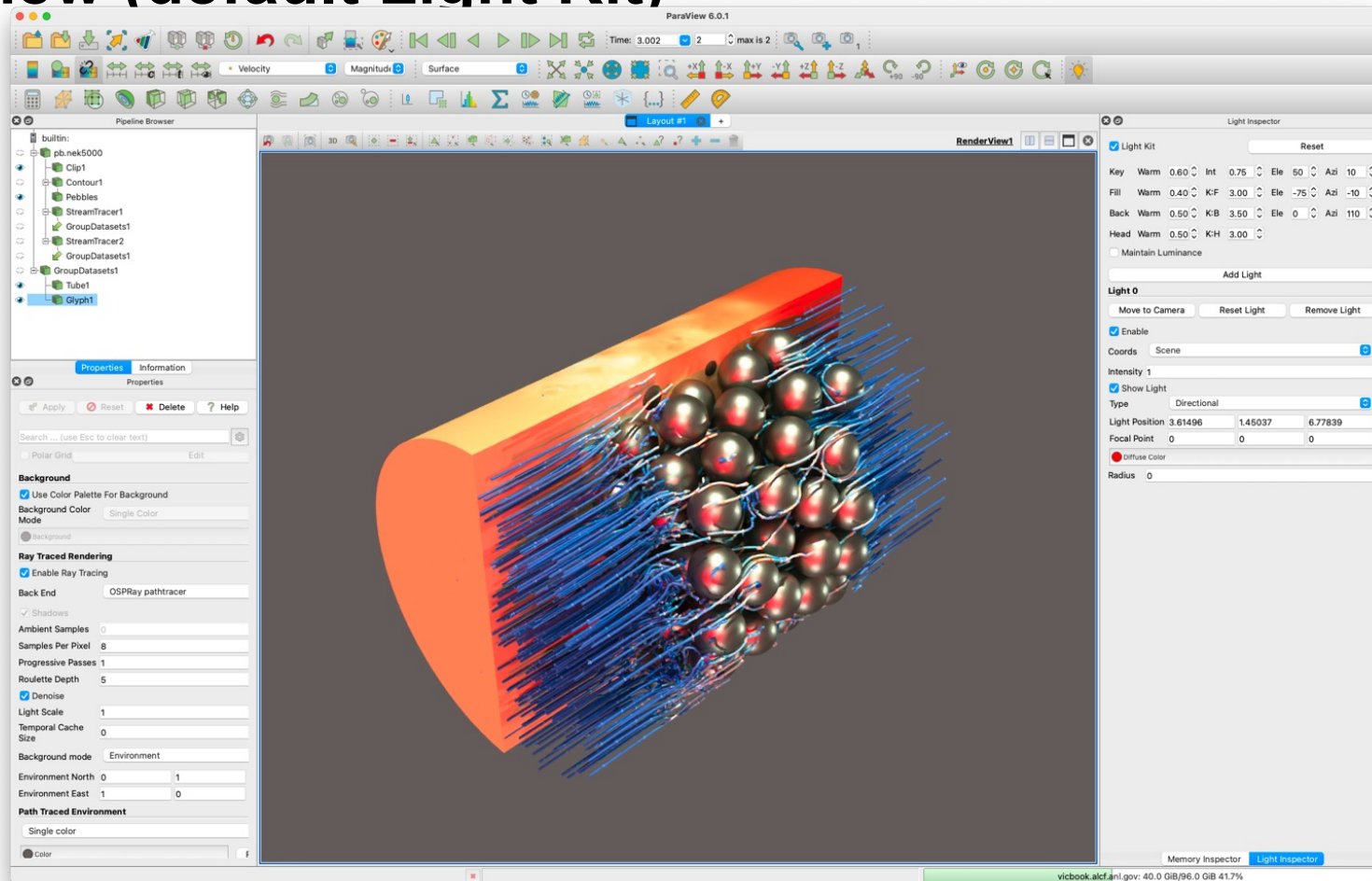


ParaView (default Light Kit)

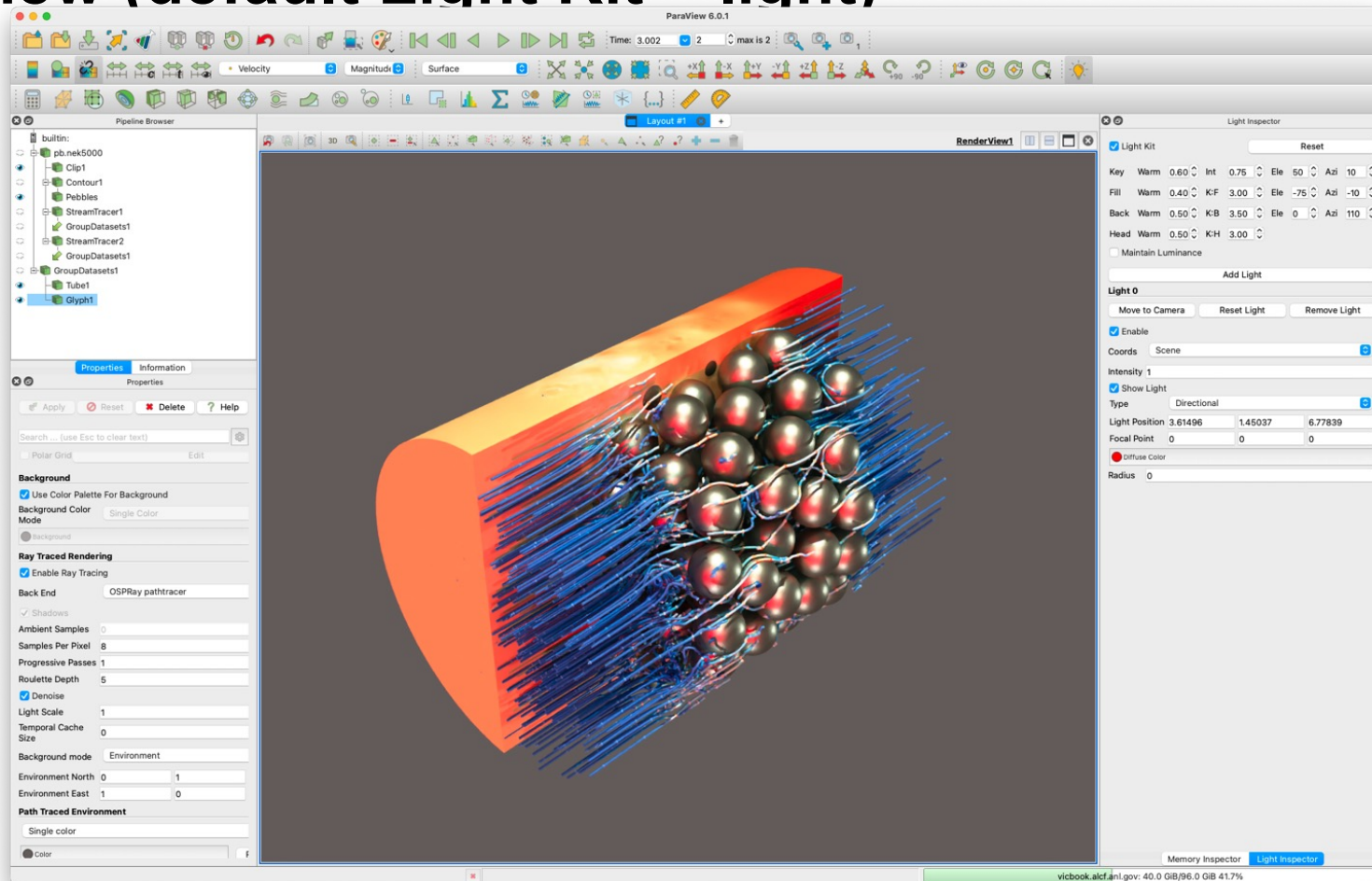
! For easier placement/best performance turn ray/path tracing off when positioning lights



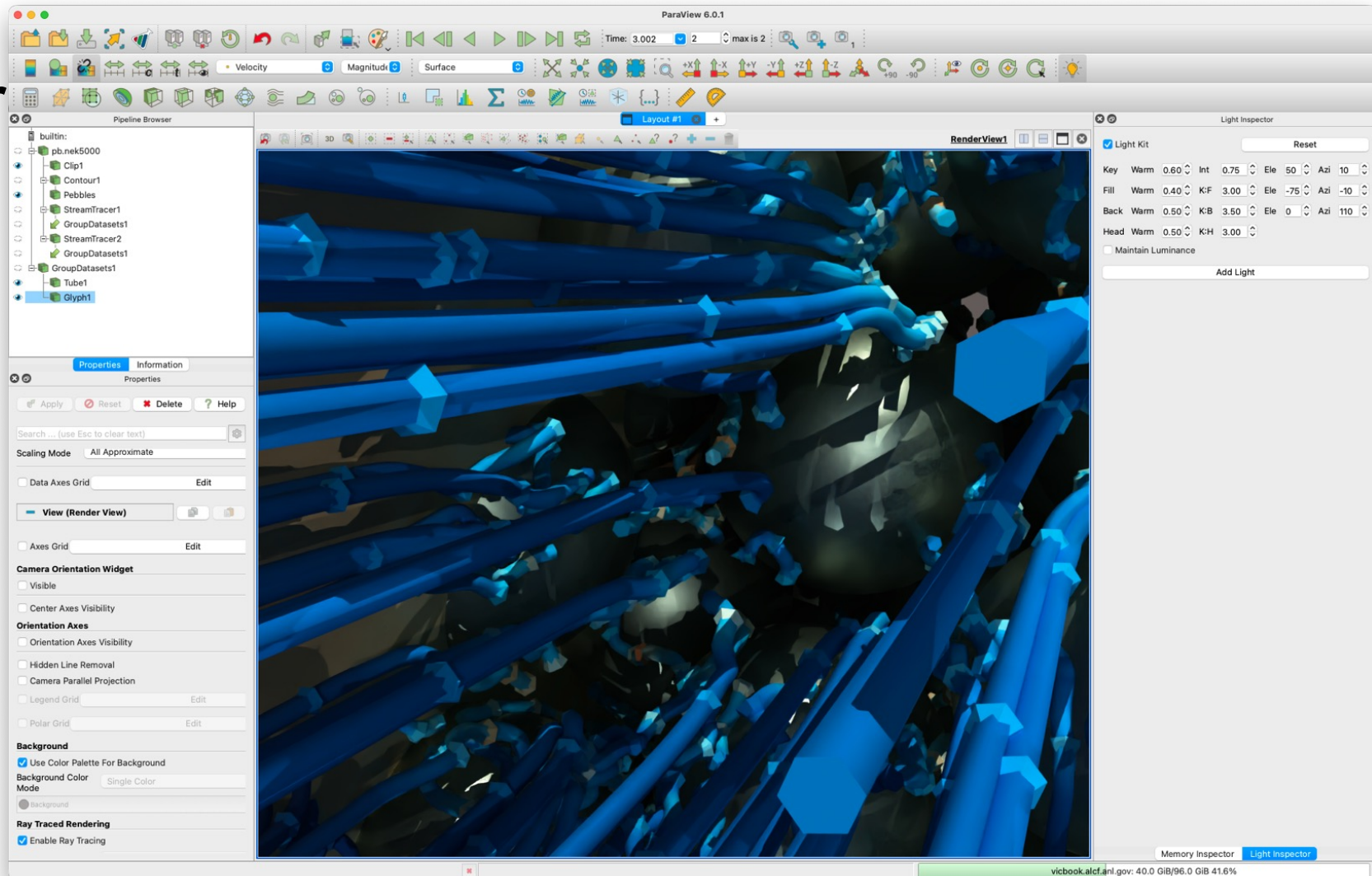
ParaView (default Light Kit)



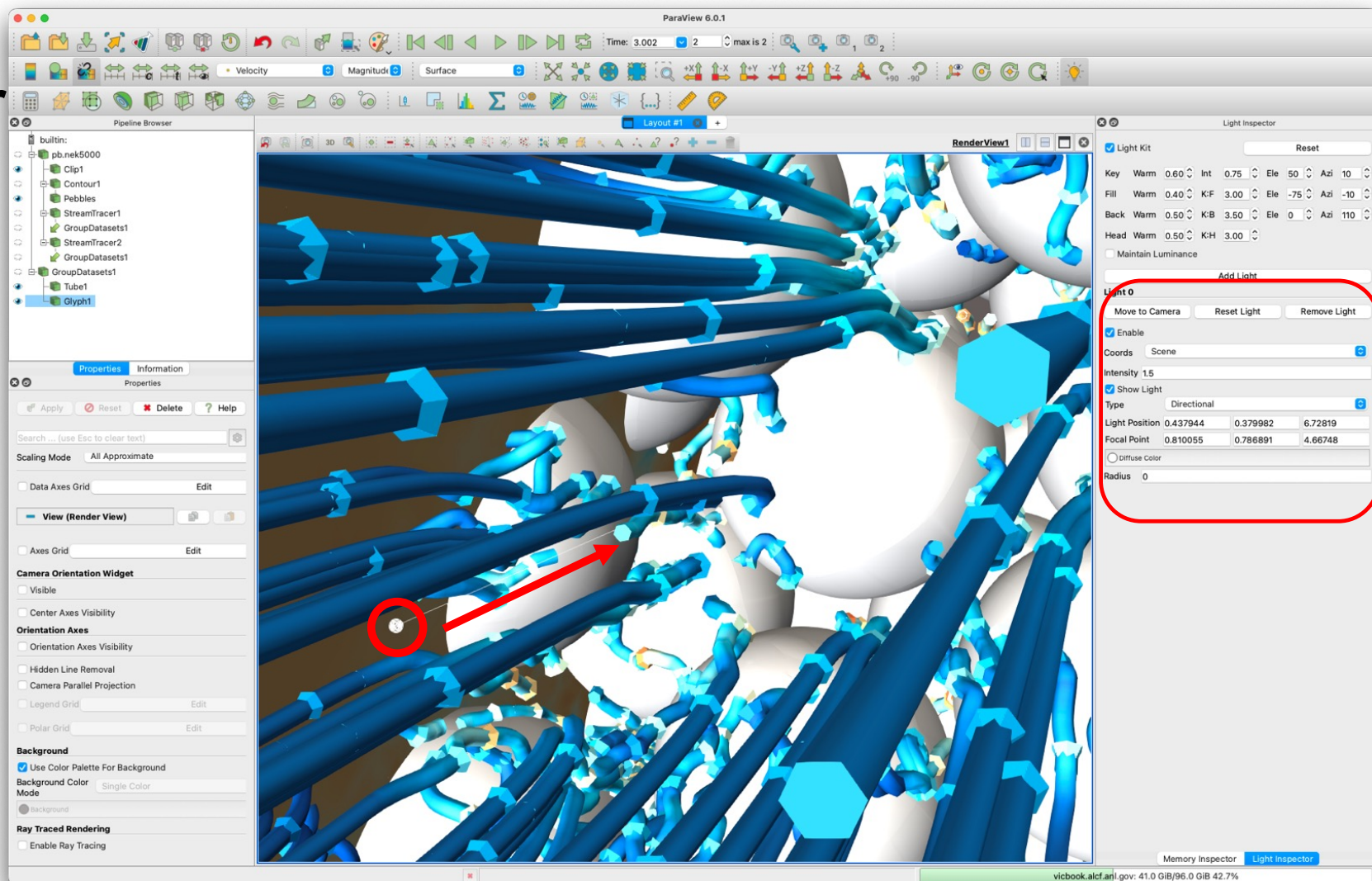
ParaView (default Light Kit + light)



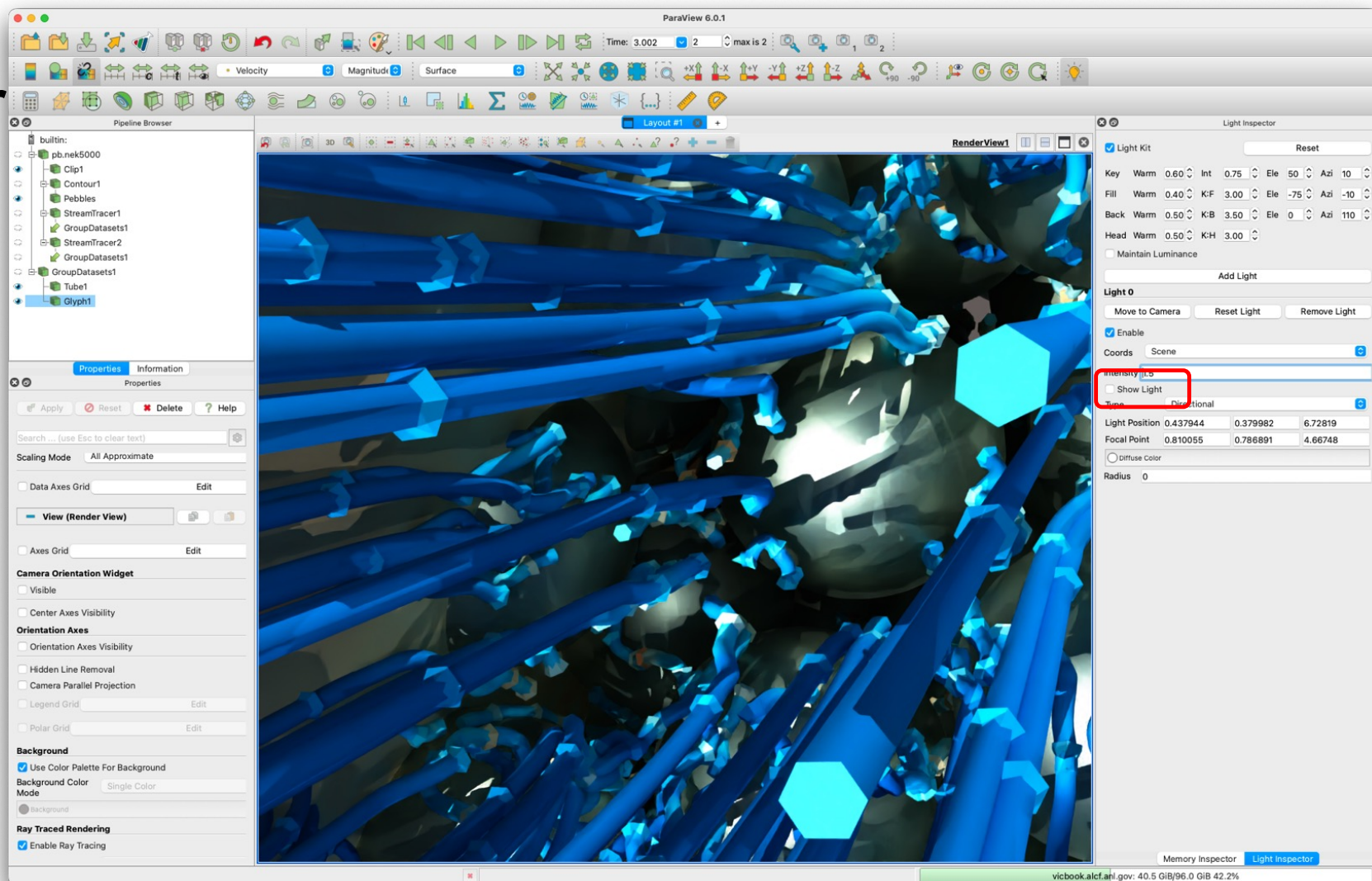
Par



Par



Par



In Situ Visualization and Analysis

In Situ vis and Analysis Problem:

- FLOPS to I/O Bottleneck

- Frontier

- Peak Performance: 1.6 EF
 - Storage: 2-4x Summit's I/O 2.5TB/s. At best 10TB/s
 - 5 orders of magnitude difference

- Aurora

- Peak Performance: 1.012 EF
 - Storage: 31TB/s
 - 5 orders of magnitude difference

Problem

- I/O is too expensive
- Scientists cannot save every timestep, and/or resolution
- Lost cycles: simulation waits while I/O is happening
- Lost discoveries: scientists might miss discoveries
- Solution: *In situ* visualization and analysis

What is *In Situ* vis and analysis

- Traditionally visualization and analysis happens post hoc
 - E.g.: Data gets saved to the disk, scientist opens it after the simulation has ended
- *In situ*
 - Data gets visualized/analyzed **while** in memory.
 - If zero-copy used, there is no data movement
 - Ideally the data is on the GPU and stays on the GPU

What is supported on ALCF resources?

- Ascent
 - GPU->GPU zero copy
 - Lightweight and fast
- How?
 - You will need to instrument code with ascent
 - Yaml file for scene/analysis/rendering definition
- Features
 - Triggers (render only when velocity > 10)
 - Python code execution
 - ML workflows
 - Rendering workflows
 - Bi-directional workflows

Ascent

Code instrumentation example

```
Ascent ascent;
ascent.open(opts);

while(running)
{
    step_sim();

    Node mesh = build_blueprint(sim_data); // zero-copy if possible
    ascent.publish(mesh);

    Node actions = build_actions(); // pipelines + scenes
    ascent.execute(actions);
}

ascent.close();
```

Visualization example

```
- action: add_scenes
  scenes:
    s1:
      plots:
        p1:
          type: pseudocolor
          field: temperature
          image_name: "out_%05d.png"
```

In situ on ALCF resources

- Ascent is installed on polaris/aurora with GPU acceleration
 - Polaris:**
module use /soft/modulefiles
module load visualization/ascent
 - Aurora:**
module use /soft/modulefiles
module load ascent
- NekRS is instrumented with ascent
 - <https://github.com/argonne-lcf/nekRS/tree/aurora-dev-insitu>
 - CPU concurrent version
- We can help!!
 - Bring us your code, we will help you instrument it!

QUESTIONS?

Joe Insley
insley@anl.gov

Silvio Rizzi
srizzi@anl.gov

Victor Mateevitsi
vmateevitsi@anl.gov

Janet Knowles
jknowles@anl.gov

Mengjiao Han
hanm@anl.gov

www.anl.gov